ITHICA: Intra-Thread Instruction Checking Approach for Defect-Induced Silent Data Corruptions

MICRO 2026 Submission **#1700** – Confidential Draft – Do NOT Distribute!!

Abstract

Hyperscalar reports of silent data corruptions (SDCs)—presumed to be caused by silicon manufacturing defects—have motivated the development of functional tests for detecting defective CPUs and their use in hyperscalar fleet studies. Interestingly, all seemingly assume that defects induce *consistent* errors, where two instances of the same instruction within the same thread, given the same architectural inputs, *always* produce the *same* wrong architectural output. We find this assumption restricts which programs can serve as tests, biasing which errors are triggered and detected, and limits identification of affected instructions to errors reproducible by short, targeted tests, biasing how errors are characterized.

We present ITHICA, which generates functional tests for defect-induced errors from arbitrary programs by inserting intra-thread, instruction-level error checks, primarily leveraging instruction duplication and output comparison. Our key insight, challenging the assumption above, is that the most pernicious defects—those most likely to escape manufacturing testing—cause *inconsistent* errors: two executions of the same instruction, given the same inputs, can produce *different* architectural outputs depending on the execution context in which they run. By exploiting this insight, ITHICA enables arbitrary programs to serve as tests and localizes affected instructions upon detection, overcoming both limitations of prior functional tests. We use ITHICA to transform industrial hyperscalar tests (our baseline), datacenter programs, and common libraries into functional tests, and evaluate them on over 3,000 CPU servers. ITHICA checks detect 39% more defective servers than baseline checks and yield novel findings on defect behavior that challenge conclusions drawn by prior hyperscalar fleet studies.

1 Introduction

Hyperscalars are reporting *silent data corruptions* (SDCs)—presumed to be caused by silicon *manufacturing defects*, resulting in permanent (hard) faults—as a critical threat to datacenter reliability [15, 16, 21, 30, 53, 82, 83]. Such SDCs occur when a *hardware error* (§2) causes a system to output an incorrect result without any indication that the error occurred [11]. Hyperscalars are finding that defect-induced SDCs impact a substantial fraction of deployed hardware, roughly one silicon device per thousand [15, 16, 30, 53, 82].

To detect defective hardware, hyperscalars have begun subjecting their server fleets to frequent functional testing. This testing has enabled hyperscalers to publish several fleet-wide studies characterizing the SDC problem [12, 15, 16, 30, 53, 82, 83], but these efforts rely on proprietary, hyperscalar- or vendor-specific tests. While some tests have been open-sourced by hyperscalars and vendors [23, 33, 51], these public suites are generally smaller in scope, and vendor tests especially tend to be structurally distinct from proprietary variants, making published findings from fleet studies difficult to replicate, validate, or improve.

Two bodies of work attempt to overcome the limitations of public functional test suites through automated test generation. The first generates tests that expose defects as captured by a specific fault model on a specific hardware implementation [46], but is limited by the inaccuracy of current fault models for defects [39, 50, 53] and the scalability challenges of fault-model-driven test generation [36, 68]. The second employs fuzzing to generate tests that thoroughly exercise a specific hardware implementation [34, 35, 37]. Both approaches generally require below-ISA hardware specifications, which are hardware-vendor-proprietary and unavailable to end-users including hyperscalers, making them inapplicable for independent, in-datacenter fleet evaluation [37]. The one exception is SiliFuzz [37], which operates at the ISA level, but generates short tests that have been deemed less effective than proprietary tests [53], a finding we corroborate in §8.

The case for programs as tests. Exercising hardware diversely is essential for triggering defect-induced errors, but without below-ISA specifications end-users cannot control or measure hardware coverage [37]. Instead, this paper proposes heuristically approximating this goal by *deriving functional tests from arbitrary programs*. This strategy is well motivated: real datacenter programs expose defects that synthetic tests miss [17, 53], by exercising complex microarchitectural features needed to trigger defects in ways that are difficult for synthetic tests to imitate [12].

The barrier imposed by existing checks. The key to transforming arbitrary programs into functional tests is instrumenting them with checks for defect-induced errors. Looking to recent work that describes or publishes functional test content [12, 15, 16, 23, 30, 33–35, 37, 51, 82, 83], three methods are used: comparing (i) inputs and outputs of invertible computations (e.g., encryption followed by decryption) [23, 33], (ii) outputs of the same computation run on different threads/cores [12, 23, 30, 33, 37]; and (iii) the output of a computation to a golden value computed on presumed-healthy hardware [12, 15, 16, 33–35, 37, 51, 82, 83]. Notably, none of these methods compares the architectural outputs of the same instruction given the same architectural inputs within the same thread. This seems to reflect a shared implicit—or explicit [51]—assumption that defects induce *consistent* errors: two instances of the same instruction given the same inputs within the same thread *always* produce the *same* wrong output.

This assumption severely limits which programs can serve as tests. Few composed of invertible computations, and most lack well-defined final outputs for golden-value or cross-thread/core checks. Even when such outputs exist, these checks require deterministic execution, making them impractical for programs involving multi-threading, network I/O, or non-determinism. These challenges are amplified if finer-grained (i.e, more frequent) checking is required in order to improve coverage (e.g., reduce logical error masking) or localize errors to instructions. For this reason, prior work typically relies on short tests for instruction localization [12, 16, 82, 83].

This Paper: Enabling programs as tests with ITHICA. Instead, ITHICA Intra-THread Instruction Checking Approach) is an automatic approach and tool that generates functional tests from programs by instrumenting them with *intra-thread*, *instruction-level* checks, primarily leveraging *instruction-duplication and output comparison* [31, 42, 43, 58, 59, 69, 72]. These checks eliminate the need for final program outputs and significantly reduce determinism requirements, enabling many more programs to serve as tests. Their instruction granularity makes them less susceptible to error masking and enables identification of failing instructions upon detection.

ITHICA's checking approach is enabled by our **key insight**: the most *pernicious* defects—those most likely to escape manufacturing testing into production—cause *inconsistent* errors: two instances of the same instruction, given the same inputs, can produce *different* architectural outputs. Intuitively, such inconsistencies result from extensive microarchitectural and electrical state in modern processors, which creates a highly diverse execution context for each dynamic instruction instance, thereby modulating how defects affect each instance architecturally. Unlike consistent errors that can be exposed from *any execution context* given the right inputs, inconsistent errors are sensitive to specific contexts drawn from an overwhelming space of possibilities (§3). Notably, inconsistent errors can arise even when the defect affects both instruction instances. On one server, two identical instructions that strongly appear to interact with the same faulty unit produce *different wrong outputs* (a case of inconsistent error), enabling ITHICA to detect the error even without a correct output reference (§7.5).

Our **first contribution** is ITHICA itself. To our knowledge, it is the first work to validate that defects cause inconsistent errors on real defective hardware and to exploit this phenomenon for defect detection. ITHICA primarily repurposes intra-thread, instruction-level checking techniques from prior work on soft error detection [58, 59, 69] and post-silicon validation [31, 42, 43, 72]. Additionally, it introduces a novel technique that proactively diversifies the execution context of memory instructions, by encouraging them to interact with different levels of the memory hierarchy. Implemented as LLVM compiler passes, ITHICA applies to arbitrary programs across multiple architectures. By relying neither on proprietary specifications nor fault models, it bridges the research gap between state-of-the-art functional tests deployed in datacenters and what is publicly available to researchers.

Our **second contribution** is a large-scale testing campaign that demonstrates ITHICA's effectiveness. From an industrial fleet of multiple millions of CPU servers, over 3,000 suspect- and confirmed-defective servers—spanning at least 10 microarchitectures across all established CPU vendors—were identified and subjected to multiple ITHICA test types: derived from representative open-source hyperscalar tests [23] (our main baseline), fleet-representative workloads [25], and common libraries [3, 40, 41, 61, 85]. We detect and analyze in detail 100 defective servers—more than 3× and 5× the two prior most detailed studies [82] and [51], respectively. By exploiting inconsistent errors, ITHICA checks detect 39% more defective servers than native final-output checks within the same hyperscalar test binaries, and 69% more than SiliFuzz [37], the only other functional test generation approach operating at or above the ISA level. ITHICA is the first technique directly compared against open-source hyperscalar tests using real defective hardware.

Our **third contribution** is a set of novel findings on defect behavior uniquely enabled by ITHICA's instruction-level checking across a variety of programs and high number of detected servers. Many of them challenge conclusions drawn by recent hyperscalar studies: (1) Since ITHICA can uniquely perform *instruction localization* (i.e., identify which instructions a defect impacts) within arbitrarily long tests, we find that the *sequence-driven execution context* in which a vulnerable instruction executes is the primary predictor of error manifestation. In contrast, *instruction stress* (i.e., how frequently a test executes a failing opcode) [82, 83] and *culprit inputs* are unreliable predictors (§7.4). (2) ITHICA enables empirically demonstrating the extreme difficulty of reproducing errors with short tests (§7.5): among the defective servers we analyze, *only one reproduces errors with single-instruction tests*; the remainder require sequences ranging from modestly longer to full programs. Prior work characterizes error trends based primarily on those reproducible with short tests, biasing error characterization toward the few cases for which short sequences are effective [51, 82, 83]. (3) Since ITHICA enables testing a wide variety of instructions in diverse execution contexts, we find that same defect often causes errors *across multiple instruction types at markedly different rates* (up to six orders of magnitude). In one such case, a server's sole detecting industry baseline test targets vector instructions, yet ITHICA also reveals failing non-vector instructions within the same test (§7.6). This finding cautions against broad claims of *hardware localization* (i.e., attributing defects to hardware components) with functional tests in general, especially those that target few instruction types [51, 82, 83].

Our findings suggest effective SDC testing requires instruction-level checking within long, diverse programs. The ideal—exercising every instruction in every possible execution context—remains an open challenge. ITHICA takes a meaningful step towards it: by relaxing the consistent-error assumption, it opens up a vast space of previously unusable programs as tests, each exposing instructions to execution contexts previously inaccessible to checking.

2 Background

Hardware Faults. Hardware faults are physical flaws or malfunctions. They can be permanent or transient.

Permanent (hard) faults include manufacturing defects, aging-related wear-out, and design bugs.

Manufacturing defects are flaws introduced during manufacture of silicon chips, which typically impact different physical regions of each affected chip in different ways. A special category of defects is *early-life failures* (ELFs), which are characteristic of weak chips that pass pre-deployment testing [53, 70], but induce erroneous behaviors within the first few weeks to months of deployment [10].

Distinct from defects (*extrinsic* flaws) [70], *aging-related faults* are caused by *intrinsic* failure mechanisms (e.g., Bias Temperature Instability (BTI) [67] or Hot Carrier Injection (HCI) [77]), which result from chip wear-out over time. Designs incorporate voltage and speed margins into chip designs to prevent errors from aging faults during design lifetimes [4].

Finally, electrical and logic *design bugs* are flaws in a *design*; thus, they affect most or all chips in a manufacturing batch. Electrical bugs manifest under specific operating conditions, namely voltage,

frequency, and/or temperature [66]. Logic bugs are flaws in a chip's logical implementation.

Transient faults occur when energetic particles, such as neutrons from cosmic rays [84] or alpha particles from packaging materials [48], generate electron-hole pairs in semiconductor devices. The resulting charge can accumulate at a transistor's source or diffusion nodes and flip the state of a logic device, a phenomenon referred to as a *transient (i.e., soft)* error. Transient faults are *not* the result of permanent hardware flaws [55].

Symptoms of Hardware Faults. A hardware fault can induce a *hardware error*, i.e., an incorrect bit stored in a flip-flop. Whether a permanent fault manifests as a hardware error may be: *sequence-dependent*, i.e., dependent on the order of stimuli; or *timing-dependent*, i.e., a subset of sequence-dependent that depends on the speed of stimuli [38]. Timing-dependent faults are dependent on a chip's electrical state (i.e., voltages and currents at transistor terminals) and thus influenced by external factors like power supply noise, frequency or temperature variations [6, 13].A hardware error may or may not manifest as *architectural error*, e.g., an incorrect architectural state (§3). If it does not, the hardware fault is said to be *architecturally masked* [75] or *benign* [54, 63]. Architectural errors can present with various symptoms, including system crashes, system hangs, or silent data corruption (SDC) [17, 53]. An SDC occurs when an architectural error causes the system to output an incorrect result without any indication that the error occurred [11]. Some architectural errors may not affect the system output at all due to *logical masking* [54, 56, 63, 75], e.g., if the error is multiplied by zero, or a program's output does not depend on it.

Testing Chips for Defects. The SDC phenomenon recently reported by hyperscalars is generally attributed to defects [15, 16, 30, 82, 83]. Standard and emerging approaches for detecting defective chips are *manufacturing* and *in-datacenter* testing, respectively.

Manufacturing testing follows chip fabrication and subjects each chip to scan-based *structural tests* [18] and (non-scan) *functional tests* [71]. Scan-based testing uses *design-for-testability* (DFT) features to apply automatically generated test stimuli to logic circuits and inspect their outputs, leveraging fault models and test metrics for systematically generating scan tests [26, 29, 47, 49, 73, 76, 79, 81]. Functional testing complements scan testing, but lacks scalably computable, high quality coverage metrics [36, 39]. Notably, manufacturing testing is highly limited in duration due to cost; exhaustive testing approaches are infeasible without reducing test volume [39].

Historically, in-datacenter testing concluded with software burn-in testing [15] (different from hardware burn-in [78]), assuming hardware faults would be caught from built-in error detection mechanisms. Upon finding that defect-induced SDCs impact a substantial fraction of servers, datacenter operators have begun subjecting them to periodic functional testing (§1).

3 Motivating ITHICA: The Inconsistent Manifestation of Pernicious Permanent Faults

Our key hypothesis in designing ITHICA is that the most *pernicious* (hard to detect) defect-induced architectural errors are **inconsistent errors**, which cause some instruction, executed from two different *execution contexts* with identical architectural inputs, to produce *different* architectural outputs. An execution context denotes the entire microarchitectural (including architectural) and electrical state of a chip. We first show how we drive this hypothesis from a recent theoretical result (§3). We then build intuition for how permanent faults lead to inconsistent errors with an illustrative RTL-level fault injection example in simulation (§3).

Architectural Errors: Formal Classification . Recent work on formal pre-silicon verification of processors and accelerators (to detect logic bugs, a type of permanent fault, §2) proves that a hardware error can manifest as one of three types of architectural errors [7, 45]:[1] (1) an **inconsistent error**, defined above; (2) a **consistent error**, which causes some instruction, executed from every execution context, to always produce *the same* wrong output for specific architectural inputs; or (3) an **unresponsive error**, which causes some instruction, executed from one context, to fail to produce an output within the expected time frame. This result holds regardless of how the hardware error arises, e.g., even if it is caused by a fault that is both timing- and sequence-*independent*. Now, consider such a fault alongside one that is timing- or sequence-*dependent*, where both faults are capable of inducing the same hardware error. That is, the first fault induces the hardware error persistently, while the second does so only under specific conditions. If the error manifests as inconsistent or unresponsive in the first case, it will do so in the second. If the error manifests consistently in the first case, it will newly manifest inconsistently in the second.

ITHICA's driving insight is that faults that induce consistent errors are easier to detect than those that induce inconsistent errors. This is because consistent errors can be exposed by executing instructions with diverse inputs from *any* execution context, while inconsistent errors require executing instructions in *many* distinct execution contexts, drawn from an *overwhelming* space of possibilities. ITHICA tests are explicitly designed to detect inconsistent errors and can implicitly detect unresponsive errors, though not consistent ones. This tradeoff pays off empirically: ITHICA detects more defective hardware than state-of-the-art tests that do not exploit its insight (§6.2).

Illustrative RTL Fault Injection Example. To build intuition for the theory in §3, we present one pedagogical RTL-level fault injection example focusing on a timing- and sequence-independent fault—the harder case for ITHICA (§3).

Our fault injection targets the open-source RISC-V CVA6 processor [60], a 64-bit, 6-stage, single-, in-order-issue core with speculation. We inject a single (i.e., one-bit) stuck-at fault [50], which is the quintessential permanent (i.e., for all clock cycles) fault, into CVA6's SystemVerilog RTL, specifically to the `rs1_valid_o` signal, which informs the next instruction to issue that its first operand (`rs1`) is valid. We then run a simple ITHICA test on the "faulty" design in Verilator simulation [74]. The test is C program of inline assembly that features two multiplications (`MUL`) using the same architectural inputs, preceded by initialization instructions as shown below:

```
lw a5, %[bval] // Sets value for a5
lw a4, %[aval] // Sets value for a4
addi t0, a5, 0 // Dummy Instruction
mul %[s1], a4, a5  // Original MUL instruction
mul %[s2], a4, a5  // Duplicated MUL instruction
```

[1] Inconsistent, consistent, and unresponsive errors map to *functional consistency*, *single-action correctness*, and *response bound* bugs in prior work [7].

We inspect the MULs' outputs once the simulation has run to completion. By forcing rs1_valid_o permanently high, both MULs are forced to perform forwarding of their rs1 operand value, without explicitly waiting for their producer to complete. Getting the value from the register file is not permitted, due to the presence of the in-flight lw producer (line 2) that writes to the same operand. As a result, the first MUL forwards a reset value (zero) for rs1, that was written in the scoreboard when the lw instruction issued. The second MUL forwards the correct value from the lw (line 2), which has completed. Note that the exact same effect is achieved if the *dummy* instruction is placed between the two MUL instructions instead.

Notably, the microarchitectural simplicity of CVA6 imposes a **more challenging case** for ITHICA: such simple core offers *less* opportunity to observe inconsistent instruction behaviors compared to more complex processors. Any increase in microarchitectural complexity makes inconsistent behaviors easier to observe, e.g., a processor with multiple execution units of the same type may dispatch the original and duplicate instructions to different units. Execution unit redundancy aids ITHICA but is not a requirement; in our real hardware results, we demonstrate a case of a failing instruction exhibiting inconsistencies detected by ITHICA despite very likely being scheduled to a single execution unit (§7.5).

Execution Context Diversity. The key to exploit microarchitectural/electrical variability for defect detection is to execute the same instruction across *diverse execution contexts* (§3). ITHICA achieves this via all available ISA-level levers: (a) varying input programs (§5.1) and transformations (§4.1), (b) varying interleaving (§4.2) to exploit **natural diversity**, where the execution context passively evolves over time between the execution of an original and duplicate instruction and (c) introducing **proactive diversity** via the novel MemDiv transformation (§4.1), which deliberately perturbs microarchitectural state, and thus execution context, by inserting additional instructions between the original and duplicate.

4 ITHICA: Intra-THread Instruction Checking Approach for Defect Detection

This section describes ITHICA's transformations (§4.1), configuration options (§4.2), and details on its LLVM implementation (§11.1).

4.1 ITHICA Program Transformations

ITHICA[2] takes as input a program and configuration options and outputs a functional test, produced by applying one or more of its transformations to the program. Each transformation inserts additional program instructions to check for errors in the outputs of specific *original* program instructions, including: (i) *validation* instructions, which perform redundant computations; (ii) *check* instructions, which detect output inconsistencies between original/validation instructions; (iii) *error-reporting branches*, which redirect control flow to an error-reporting basic block when an output inconsistency is detected; and (iv) novel *diversity* instructions, which create proactive diversity in microarchitectural state between the execution of original and validation instructions.

The first three transformations below are adapted from prior work on post-silicon validation for design bugs [31, 42, 43] and soft error detection [58, 59, 69]. As they were not publicly available and described in assembly, we re-implement them for LLVM IR (§11.1).

Arith instruments arithmetic instructions, including, among others, logic, binary, vector and conversion operations [44] (detailed in [5]). Validation instructions duplicate the original ones and their results are checked for mismatches against them. Duplicates preserve original data dependencies, allowing error propagation even when comparisons are inserted less frequently (§4.2).

Mem instruments load and store instructions. Loads are duplicated and their outputs are compared against the originals. For stores, validation instructions are loads from the store's address, and their outputs are compared against the original store's value operands. Atomic and volatile instructions are excluded from instrumentation to maintain correctness.

Br instruments conditional branches to verify that they resolve to the correct target by recording the *intended* target address before the branch resolves, and validating it against the *actual* address taken. Br is more lightweight than prior work [14, 27, 58, 69]: it exclusively targets conditional branches, and focuses on wrong-target misdirections between two valid targets, which are more likely to manifest silently. Branching to a rogue address is more likely to result in a non-silent architectural error, like a crash (§2), which would be detected without an instruction-level check [9].

A limitation of Mem, as implemented by ITHICA and prior work [42, 43], is that validation loads typically retrieve data: via an L1 cache hit (for checking loads), possibly missing errors in load outputs during L1 misses; or a core-local store buffer (for checking stores), possibly missing errors in store outputs at lower memory hierarchy levels. To address this and increase coverage, ITHICA introduces the novel MemDiv transformation, representing the first use of *proactive diversity* (§3) to check for errors affecting memory instructions.

MemDiv extends Mem by introducing diversity instructions, namely *mfence* (*memory fence*) and *clflush* (*cache line flush*). These instructions flush store buffers and caches by virtual address, respectively, encouraging original and validation memory instructions to interact with and exercise defects within different levels of the memory hierarchy. Specifically, *mfence* encourages retrieval via an L1 cache hit. The subsequent *clflush* forces data retrieval directly from main memory. Note that parity/ECC might not cover all data movement of a complex memory system, nor be able to detect all errors [52].

4.2 ITHICA Configuration Options

ITHICA supports configuring the location of validation instructions and the insertion frequency of comparison instructions.

Configuring Interleaving. ITHICA's *interleaving* controls the insertion points of validation/diversity instructions relative to originals. An interleaving of n indicates that n original instructions will appear consecutively in program order before their corresponding validation/diversity instructions appear in the same order.

Configuring Block Size. ITHICA's *block size* controls the insertion frequency of comparison instructions. A block size of n indicates that comparisons will be inserted for every n-th original instruction. Validation instructions are inserted for *all* relevant original instructions, ensuring that errors impacting validation instructions can propagate through dependency chains to comparison

[2] We will make our anonymous repository [5] public upon publication.

	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	D11	D12	D13	D14
***μ*arch**	u1	u2	u3	u4	u5	u5	u2	u6	u1	u4	u3	u5	u1	u2
CPU Age	19	6	34	16	45	38	81	19	24	11	11	42	10	64

Table 1: DPool characteristics. CPU age is shown in months.

instructions (unless they get logically masked §2). Neither interleaving nor block size is configurable for `Br`, as it instruments only the single original conditional branch in a basic block, if one exists.

4.3 ITHICA LLVM Implementation

We implement each of the four ITHICA transformations, as well as their combinations (`Arith+Mem`, `Arith+MemDiv`, `Arith+MemDiv+Br`) as distinct LLVM IR compiler passes. Each pass can be configured with different block size and interleaving values via command-line options. Implementing ITHICA at the LLVM IR level provides portability, as IR passes are target-agnostic and can be integrated into existing build systems without requiring modifications to the compiler. Due to space limitations, we omit details of our implementation but include them in our anonymous repo for reference [5].

We apply ITHICA as the final IR-level pass, to minimize interference with compiler optimization passes. Despite that, backend compiler optimizations can still eliminate our inserted instructions. To prevent this without modifying compiler internals, we mark duplicated loads and stores as volatile. For arithmetic instructions, we pass their arguments through no-op volatile store-load pairs to achieve the same effect. Future versions of Clang may offer intrinsics to preserve side-effect-free instructions more cleanly.

5 Testing Campaign: ITHICA on Real Hardware

To test our insight (§3) and evaluate ITHICA tests generated from various programs (§5.1), we conduct a large testing campaign (§5.2).

5.1 Instrumenting Diverse Input Programs

We use ITHICA to instrument three input program types: (i) industry-representative functional test programs comprising Google's **cpu-check (CC)** [23] (ii) industry-representative workloads comprising Google's **Fleetbench (FB)** [1, 25]; and (iii) **libraries** [3, 40, 41, 61, 85] used by `CC` and `FB`. When ITHICA generates a functional test from a program, we append "`-`" and then "`ITHICA`" or the name of the applied transformation (e.g., "`Arith`") to the program name to get the new test name (e.g., `CC-ITHICA` or `CC-Arith`, respectively).

cpu-check. `CC` is a suite of bespoke functional tests with final output checks (§1) for in-datacenter testing (§2). Test programs include: invertible computations in sequence (e.g., encryption-decryption) with input-output comparisons; checksum and hash computations on random, non-deterministic data with cross-core output comparisons; and AVX computations with vector lane output comparisons.

Fleetbench. `FB` consists of C++ microbenchmarks, representing hot functions in Google datacenters, including Proto, Swissmap, Libc, TCMalloc, Hashing, Compression, and STL-Cord, executed with deterministic data pre-sampled from a production distribution or randomly generated with a fixed seed. We exclude Compression due to its reliance on *openmp*, which we could neither statically compile nor access as a system library on our servers.

Libraries. For both `CC` and `FB`, we instrument both top-level and library code. For `CC`, we apply ITHICA to `libc`, `libc++`, `Abseil`, `OpenSSL`, and `zlib`. Due to tight coupling of `libc` and `libc++` with GCC, we instead apply ITHICA to LLVM's overlay libraries, `llvmlibc` and `llvmlibcpp`. For `FB`, we do not separately instrument libraries, like we do for `CC`, but rely on `FB`'s build system (Blaze) to rebuild all relevant libraries (e.g., Abseil) with ITHICA.

Metric	Description
Server Detections	# servers with error detection
Error Detection Rate (EDR)	# runs with error detect. / # all runs (%)
Error Frequency (EF)	# error detections / # all runs
Time to Detection (TTD)	time from test start to first error
PC Sensitivity *(per opcode)*	# failing static PCs / # all static PCs (%)
Input Breadth *(per opcode)*	# unique fail. inp. / # all fail. inp. (%)

Table 2: Evaluation Metrics. Each run spans one hour.

5.2 Two-Pool Evaluation Strategy

We conduct experiments on two groups of CPU servers drawn from a hyperscale fleet of multiple millions of servers: the **Quarantine Pool (QPool)** and the **Defective Pool (DPool)**.

The QPool consists of more than **3,000 servers** quarantined by the hyperscalar for out-of-production testing due to suspicious behavior. They span **at least 10 microarchitectures** across all CPU vendors. It is a *dynamic* pool (i.e., new servers are regularly added and removed) containing both suspect- and confirmed-defective servers. Servers are offlined to the QPool via three mechanisms: (1) customer complaints about data corruption; (2) opportunistic screening with hyperscalar- and vendor-developed tests; and (3) forensics-based analysis of hardware and software exceptions (e.g., fail-stop and fail-slow errors including machine check exceptions, kernel or user crashes, invariant fails in user code).

The DPool consists of 20 confirmed-defective servers, originally part of the larger QPool, that set aside for dedicated testing with ITHICA. All 20 have failed at least one functional test developed either by the hyperscalar or by the CPU vendor and exhibit diversity in terms of microarchitecture, geographic location, age, and time in the QPool (Table 1). Across all our experiments, no functional test (i.e., neither ITHICA nor `CC`) detects any error on six of them. Thus, we report results for the *14 remaining servers*.

Benefits of Two-Pool Evaluation. The DPool and QPool serve distinct roles. The DPool provides a controlled environment with confirmed-defective servers, enabling us to evaluate our hypothesis that intra-thread, instruction-level checks can detect defect-induced errors. Its static nature and our exclusive access to it ensure sufficient runtime to evaluate all ITHICA transformations and configurations: we collect **over 2,000 hours of testing time per DPool server**, exceeding the allocated time of previous detailed studies by 1-2 orders of magnitude [51, 82, 83]. In the QPool, ITHICA tests integrate into the datacenter operator's standard out-of-production testing pipeline alongside other industrial tests, enabling large-scale evaluation in a real-world in-datacenter setting.

6 Experiments Overview

The experimental evaluation in this paper is partitioned into two sub-sections: testing on the DPool (§6.1) and testing on the QPool (§6.2). Across our experiments, we compare the efficacy of ITHICA tests to `CC` (our baseline) using four main metrics, defined in the first four rows of Table 2. The results are interpreted in detail in §7.

	D1			D2			D3			D4			D5			D6			D7			D8			D9			D10			D11			D12			D13			D14		
	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr	Ith	Nat	Cr
Arith	100 (9)	0 (0)	0	0 (0)	37 (1)	0	43 (3)	39 (3)	2	89 (4)	4 (4)	11	3 (0)	0 (0)	0	0 (0)	0 (0)	0	97 (91)	70 (66)	3	2 (0)	0 (0)	0	93 (0)	0 (0)	0	1 (0)	1 (0)	1	88 (1)	86 (0)	3	7 (2)	0 (0)	0	0 (0)	20 (2)	12	1 (1)	0 (0)	2
Mem	0 (0)	0 (0)	0	16 (15)	31 (15)	10	50 (0)	61 (0)	1	0 (0)	97 (0)	3	0 (0)	0 (0)	0	0 (0)	0 (0)	0	0 (0)	74 (0)	26	0 (0)	0 (0)	1	0 (0)	0 (0)	3	0 (0)	0 (0)	0	3 (0)	97 (0)	0	0 (0)	0 (0)	0	0 (0)	62 (25)	12	0 (0)	0 (0)	0
MemDiv	0 (0)	0 (0)	0	0 (0)	0 (0)	28	0 (0)	0 (0)	0	0 (0)	100 (0)	0	0 (0)	0 (0)	0	0 (0)	0 (0)	0	0 (0)	100 (0)	0	0 (0)	0 (0)	0	0 (0)	0 (0)	0	0 (0)	0 (0)	0	0 (0)	45 (0)	0	1 (0)	1 (0)	0	3 (2)	1 (0)	1	0 (0)	0 (0)	0
Br	0 (0)	0 (0)	0	0 (0)	23 (0)	0	0 (0)	40 (10)	0	0 (0)	100 (0)	0	0 (0)	0 (0)	0	0 (0)	0 (0)	0	0 (0)	100 (0)	0	0 (0)	0 (0)	3	0 (0)	0 (0)	0	0 (0)	0 (0)	0	0 (0)	99 (0)	0	0 (0)	0 (0)	0	0 (0)	85 (39)	2	0 (0)	0 (0)	0
Arith+ Mem	40 (2)	0 (0)	0	9 (9)	35 (9)	25	42 (8)	42 (8)	0	95 (9)	9 (9)	5	0 (0)	0 (0)	0	0 (0)	0 (0)	0	22 (13)	17 (9)	58	0 (0)	0 (0)	3	85 (1)	0 (0)	5	0 (0)	0 (0)	0	77 (0)	79 (0)	8	4 (3)	0 (0)	0	0 (0)	8 (0)	14	0 (0)	1 (1)	1
Arith+ MemDiv	10 (1)	0 (0)	0	0 (0)	0 (0)	28	70 (3)	10 (3)	0	100 (2)	82 (0)	0	0 (0)	0 (0)	0	0 (0)	0 (0)	0	100 (100)	100 (100)	0	0 (0)	0 (0)	0	100 (0)	0 (0)	0	0 (0)	0 (0)	0	57 (0)	59 (0)	0	3 (0)	0 (0)	0	0 (0)	0 (0)	2	0 (0)	0 (0)	0
Arith+ MemDiv+Br	10 (0)	0 (0)	0	0 (0)	0 (0)	10	60 (0)	10 (0)	0	100 (0)	76 (0)	0	0 (0)	0 (0)	0	0 (0)	0 (0)	0	100 (100)	100 (100)	0	0 (0)	0 (0)	1	99 (0)	0 (1)	0	0 (0)	0 (0)	0	54 (0)	58 (0)	0	0 (0)	0 (0)	0	0 (0)	0 (0)	0	0 (0)	0 (0)	0
CC	—	0 (0)	0	—	73 (6)	8	—	15 (1)	0	—	100 (0)	0	—	0 (0)	0	—	0 (0)	0	—	95 (0)	0	—	0 (0)	2	—	0 (0)	0	—	0 (0)	0	—	98 (0)	2	—	2 (0)	0	—	55 (3)	8	—	1 (0)	1

Figure 1: EDR for different CC-ITHICA tests (with block size and interleaving of 1) and CC in DPool (D1–D14). Each triplet reports results for ITHICA (*Ith*), Native (*Nat*), and program crashes with no detection (*Cr*). For *Ith* and *Nat*, the subset of executions that crashed after a detection is shown in parentheses. D6* is uniquely detected by Arith for interleaving of 8 (§6.1).

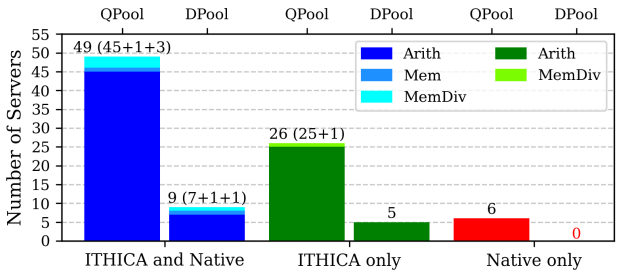


Figure 2: Detections across all CC-ITHICA runs in both pools.

6.1 Testing on the DPool

First, to compare CC-ITHICA to CC, we run **eight functional tests** on each DPool server. Seven are CC-ITHICA tests obtained by applying its four main transformations (§4.1) and three combinations thereof (Arith+Mem, Arith+MemDiv, Arith+MemDiv+Br) on CC, all with block size and interleaving set to 1. The eighth is the original CC for comparison. Each test runs 100 times per server, constituting **100 runs**, each lasting **one hour** unless interrupted by a crash. Between runs, the server is rebooted.

Within an ITHICA test, we distinguish between Native checks (checks in the original program, i.e., CC's final output checks) and ITHICA checks (inserted by ITHICA). If either check type detects an error before a crash, the detection counts. CC contains only Native checks; CC-ITHICA contains both. Each run executes hundreds of thousands rounds of the CC-ITHICA program (and more rounds for CC), each with different randomly-generated inputs, yielding hundreds of thousands of inputs per run.

Fig. 1 shows EDR for the seven CC-ITHICA tests and CC across all 14 DPool servers (D1–D14). Parenthetical values show the subset of detections followed by crashes; the *Cr* column shows crashes without any other detection.

To compare different ITHICA interleaving and block size configurations (§4.2) for the same program, we conduct two other experiments. First, fixing block size to 1, we **sweep interleavings** of 1, 2, 4, 8, and max (basic block length). Second, fixing interleaving to 1, we **sweep block sizes** of 1, 2, 4, 8, and dep, which inserts comparisons at the end of each instruction's dependency chain. For each configuration, we run 100 one-hour executions. Fig. 3 top/bottom shows the results of the block size/interleaving experiment.

Finally, we evaluate ITHICA test content generated from two types of non-test programs, using the Arith transformation: **five libraries** used by CC and **six FB workloads** (§5.1). We run each FB-ITHICA binary for 100 one-hour executions.

Ratio ITHICA/Native	Geom. Mean	Arith. Mean	Median	Min	Max
Error Detection Rate (EDR)	1.78	4.50	1.17	0.52	100.00
Error Frequency (EF)	7.51	130.15	3.44	0.02	5396.00
Time to Detection (TTD)	0.68	0.77	0.56	0.33	1.72

Table 3: Comparison of ITHICA and Native detection metrics (EDR, EF, TTD) for 58 servers detected by both methods.

6.2 Testing on the QPool

Due to limited available execution time in the QPool compared to the DPool, we primarily run **CC-Arith** with block size and interleaving of 1, as this configuration yielded the most detections on the DPool (Fig. 1, §7). The total execution time accumulated per server varies, ranging from 20 to 100 hours due to the presence of other tests in the pipeline and the dynamic nature of the QPool.

We secondarily run **CC-Mem, CC-MemDiv and CC-Br**, reporting *only their unique detections* compared to CC-Arith due to their shorter runtime. Our baseline for each remains Native checks within the same CC-ITHICA binary. To ensure reliable analysis, we exclude servers with less than 20 execution hours per test from our results. The results are summarized in Fig. 2, alongside the aggregate DPool results. Finally, as in the DPool, we evaluate CC's **five libraries** and the **six FB workloads** transformed with Arith.

7 Results

We present ten key findings derived from the experiments in §6.1 and §6.2, as well as some follow-up experiments for further analysis.

7.1 Defects Manifest as Inconsistent Errors

Obs. 1 Fig.1 shows CC-ITHICA tests together detect errors in all 14 DPool servers. Among the four base transformations, Arith is the most effective, detecting 11 out of 14 servers. This result should *not* be interpreted as evidence that instructions instrumented by Arith are inherently more vulnerable; it may reflect survivorship bias in the tests used to flag servers for offline testing (§5.2). Server D2 is only detected by Mem and Arith+Mem; both check for errors in memory instructions. Br does not detect errors in the DPool, but it flags (non-unique) servers in the QPool (§6.2). **Obs. 2** Four servers (D1, D5, D8, D9) are uniquely detected by ITHICA checks

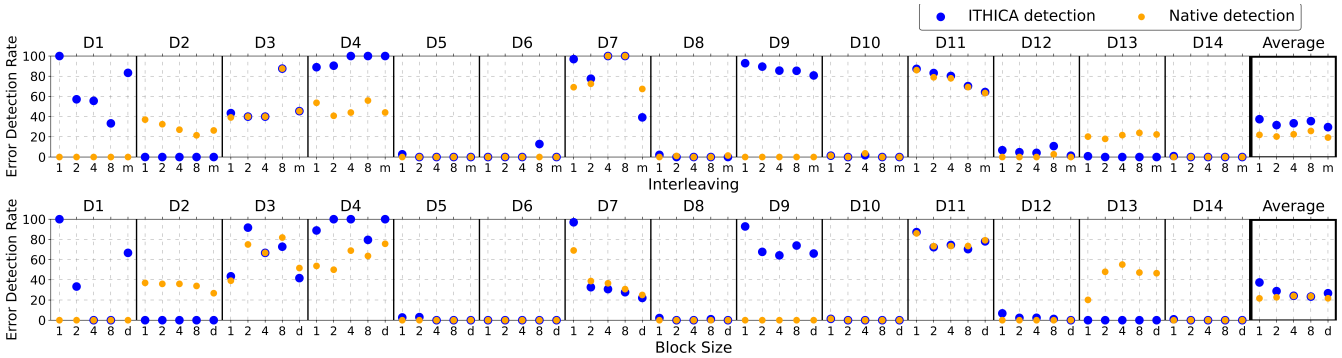


Figure 3: Impact of interleaving and block size on EDR for each DPool server (D1 to D14), for CC-Arith. The top row shows the effect of varying interleaving (m=*max*, length of the basic block), while the bottom row shows the effect of varying block size (d=*dep*, length of instruction dependency chain). The rightmost panels show the average EDR across all servers.

in CC-ITHICA tests while missed by Native checks in the same binaries, as well as by CC.

Obs. 3 As shown in Fig. 2, 49 QPool servers were detected by both ITHICA and Native checks, with 45 of them flagged by Arith, 1 uniquely by Mem, and 3 uniquely by MemDiv. **Obs. 4** An additional 26 QPool servers were only captured by ITHICA checks—25 by Arith and 1 uniquely by MemDiv. Only 6 servers were detected by Native but not by ITHICA. Two possible explanations are coverage gaps in our instrumentation and consistent manifestation of defect-induced errors for small interleaving values (§7.3), discussed in §8. Across both pools, ITHICA detects **39% more servers** than Native.

> **Finding 1:** Defects, despite being permanent faults, manifest as inconsistent errors in nearly all servers flagged as defective in our testing campaign (Obs.1-4).

For the experiment of Fig. 2, Table 3 compares ITHICA and Native checks on several other metrics (Table 2). **Obs. 5** Across 58 servers detected by both ITHICA and Native (49/9 in the QPool/DPool), ITHICA achieves 1.78× EDR improvement, 7.51× EF improvement and 0.68× TTD improvement (1.47× faster).

> **Finding 2:** ITHICA checks outperform final output checks used by hyperscalar tests across all evaluated metrics (Obs.1-5).

Fig. 4 shows the distribution of failing instructions and their average error frequencies across all CC-ITHICA tests run in the DPool with a block size of 1 and an interleaving of 1, except for D6 with an interleaving of 8 (§6.1). Cases where no specific instruction is identified (*no-instr*) can result from an error in an instrumentation instruction itself (e.g., icmp), or a crash before logging completes. **Obs. 6** The data reveals a diverse range of affected instructions across servers, spanning arithmetic, floating-point, vector, and memory instructions. Error frequencies also vary drastically, from less than one per run to over a thousand, on average. This reflects the fact that defects impact different physical regions of each chip in different ways (§2). The instruction types, frequencies, and program locations of these errors are analyzed in detail in §7.3–7.6.

While ITHICA can detect errors that manifest inconsistently regardless of their source, the observed characteristics narrow plausible fault categories. **Obs. 7** Their frequency and repeatability make transient faults less plausible (Fig. 4); their device-specific (Fig. § 4) yet microarchitecture-agnostic (Table 1) nature points away from design bugs; their presence across devices of varying ages (Table 1) challenges aging effects as a sole cause. Taken together, these observations are consistent with defects.

ITHICA Pass	Arith (Block Size) (1)	(2)	(4)	(8)	(dep)	Mem	MemDiv	Br
Performance	2.18	1.79	1.64	1.52	1.75	1.17	53.67	1.17
Binary Size	8.47	5.88	4.59	4.11	6.11	3.06	13.12	1.24

Personal, non-industrial, server — Intel Xeon Gold 6226R, 2.9GHz, 2×16 cores, 2 threads/core, L1: 1MB D+I, L2: 32MB, L3: 44MB, Mem: 500 GB

Table 4: Performance overhead and binary size increase for ITHICA transformations compared to CC.

> **Finding 3:** ITHICA-detected errors in our server pools are most consistent with defects as the underlying cause (Obs.6, Obs.7).

Implications for Chip-Users: *Intra-thread, instruction-level checking is highly effective for defect detection, contradicting the apparent implicit assumption of existing functional tests [12, 15, 16, 23, 30, 33–35, 37, 51, 82, 83]. This opens up using arbitrary programs as tests, removing the bias toward programs that satisfy the requirements that existing checking methods impose (§1).*

7.2 ITHICA Paves the Way for Online Testing

The permanent nature of defects has a direct practical implication for ITHICA instrumentation. For defect detection (i.e., identifying whether a server is defective) catching *any one* error per server suffices, unlike soft error protection which requires catching *every* corruption that affects application output. This distinction enables ITHICA instrumentation to be reduced, trading EDR for lower overhead, while maintaining high coverage (# defective servers detected). Notably, for error characterization, fine-grained instrumentation at every instruction remains necessary (§7.6).

In this work, one strategy for reduced instrumentation is explored: increasing *block size* (i.e., comparison frequency). The results in Fig. 3 show that a block size of 1 performs best on average, likely due to reduced logical error masking.

Obs. 8 However, larger block sizes demonstrate comparable effectiveness: all servers with reproducible (i.e., more than one) detections at block size 1, except D10 and D14, are also detected at larger block sizes. This indicates that checking frequency can be

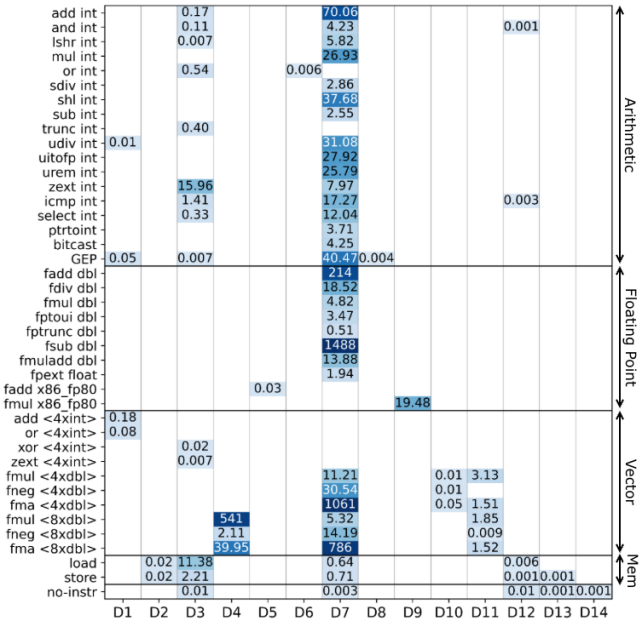


Figure 4: Distribution of instructions where ITHICA detects errors, with EF calculated across all CC-ITHICA runs in DPool.

relaxed without significantly compromising coverage. The overhead of Arith decreases from 2.18× at block size 1 to 1.5× at block size 8 (Table 4). While exploring optimal selective instrumentation strategies is beyond the scope of this work, it represents important future work towards leveraging ITHICA for online testing.

Finding 4: ITHICA instrumentation and runtime overhead can be reduced at high defect detection coverage (Obs. 8).

Implications for Chip-Users: *The permanent nature of defects can be exploited for relaxed instrumentation. ITHICA checking inherently supports this relaxation and is the key to online, in-production testing. Establishing the effectiveness of ITHICA for defects is the necessary first step towards that goal.*

7.3 Execution Context Variation is Important

Obs. 9 As shown in Fig. 1, D13 is uniquely detected by MemDiv; all other ITHICA transformations miss it. Errors are localized to the third validation load (following clflush) that checks an original store, while the previous two loads yield correct results. One additional server in the QPool is similarly detected exclusively by MemDiv. This suggests that the clflush and mfence instructions inserted by MemDiv perturb the execution context between memory instructions sufficiently to expose the inconsistency.

Fig. 3 shows that some servers exhibit improved detection with larger interleaving (e.g., D3, D4). **Obs. 10** D6 is exclusively detected at interleaving of 8. Overall, small interleavings are sufficient to detect most defective servers in the DPool.

These observations show that, for arithmetic instructions, *natural diversity* from modest interleavings is generally sufficient to expose inconsistencies, possibly because relevant execution context

Instruction with Error	Original	Validation	Both
% across all DPool runs	31.3%	44.7%	24.0%

Table 5: Decomposition of instructions exhibiting errors in DPool into original, validation (ITHICA-inserted), or both.

is largely core-side. *Proactive diversity* is necessary to expose certain defects affecting memory instructions, as demonstrated by the MemDiv detections, possibly due to their relevant execution context spanning core and uncore components.

Finding 5: The detection of defect-induced inconsistent errors depends on (typically modest) execution context variation between original and validation instructions (Obs.9, 10).

Obs. 11 Merged passes shown in Fig. 1 detect servers with errors across multiple instruction types (e.g., D1) but sometimes have lower EDR or entirely miss servers detected by individual passes. This can be attributed to the additional instrumentation instructions altering the execution context needed for error manifestation.

We also analyze which of the two instructions (original or validation) exhibits an error, reported in Table 5 across DPool runs. **Obs. 12** In 24% of cases, **both instructions produce incorrect but different outputs**. These cases are particularly informative because they constrain the possible explanations for the inconsistency: since both are incorrect, both likely touched the same defective hardware component, yet still produced different outputs. In contrast, when only one instruction is incorrect, we cannot determine whether they used the same component or were scheduled to different ones (i.e., one defective and one not). Either scenario represents an execution context difference that explains the inconsistency.

For each failing opcode-server combination, we analyze what fraction of static PCs executing the failing opcode exhibit errors to calculate **PC Sensitivity** (Table 2). **Obs. 13** For most opcodes, PC Sensitivity is very low (often <1%, Table 6), revealing that the same opcode behaves correctly in the vast majority of its program locations. A notable exception is vector double instructions, which exhibit higher PC Sensitivity. However, for each of these opcodes, all PCs are contained within the same basic block (★ in Table 6), and therefore likely execute under similar execution context.

Finding 6: Whether an instruction exhibits a defect-induced error depends on whether it executes in a vulnerable execution context, shaped by its preceding sequence (Obs.11, 12, 13).

Implications for Chip-Users: *The key to exploit microarchitectural (and resulting electrical) variability for defect detection is to execute the same instruction across diverse execution contexts. Within a given program, the available ISA-level levers are: (a) varying Interleaving to exploit natural diversity, (b) varying and combining ITHICA transformations to alter the surrounding instruction mix, and (c) using* MemDiv *for proactive diversity. We show that all influence detection.*

7.4 Program Variation is Important

Table 7 shows detected servers across top-level CC, its library code, and FB. Since CC comprises multiple subtests (§5.1), we separate results by where ITHICA detected errors. For each, we report total detections and unique detections compared to the other tests.

Category	Failing Opcode	Affected Servers	# Total PCs	# Failing PCs	PC Sensitivity Geom. Mean (Std)	# Total Failing Inputs	# Unique Failing Inputs	Input Breadth Geom. Mean (Std)
Integer Arithmetic	add int	D3	1,202	5	0.42% (0.00)	22	7	31.82% (0.00)
	and int	D3 / D12	748	3 / 1	0.23% (0.13)	2 / 1	2 / 1	100.00% (0.00)
	lshr int	D3	424	1	0.24% (0.00)	1	1	100.00% (0.00)
	or int	D3 / D6	180	7 / 1	1.47% (1.67)	776 / 9	34 / 2	9.87% (8.92)
	trunc int	D3	492	5	1.02% (0.00)	6	4	66.67% (0.00)
	udiv int	D1	36	1	2.78% (0.00)	1	1	100.00% (0.00)
	zext int	D3	809	46	5.69% (0.00)	19,708	217	1.10% (0.00)
	getelementptr	D1 / D3 / D8	5,568	6 / 1 / 1	0.03% (0.04)	3 / 1 / 4	3 / 1 / 1	63.00% (35.36)
Comparisons	icmp int	D3 / D12	3,968	12 / 1	0.09% (0.14)	17 / 1	4 / 1	48.51% (38.24)
	select int	D3	617	7	1.13% (0.00)	84	4	4.76% (0.00)
FP Arithm	fadd x86_fp80	D5	4	1	25.00% (0.00)	36	6	16.67% (0.00)
	fmul x86_fp80	D9	4	1	25.00% (0.00)	17.92M	1,330	0.01% (0.00)
Vector <4xint>	add <4xint>	D1	11	2	18.18% (0.00)	1	1	100.00% (0.00)
	or <4xint>	D1	2	1	50.00% (0.00)	16	4	25% (0.00)
Vector <4xdbl>	fmul <4xdbl> ★	D10 / D11	8	3 / 8	61.24% (31.25)	6 / 740,916	4 / 1,570	3.76% (33.23)
	fneg <4xdbl> ★	D10	8	5	62.50% (0.00)	2	2	100.00% (0.00)
	fma <4xdbl> ★	D10 / D11	8	7 / 5	73.95% (12.50)	77 / 303,198	19 / 803	2.56% (12.21)
Vector <8xdbl>	fmul <8xdbl> ★	D4 / D11	8	8 / 8	100.00% (0.00)	20.10B / 169,199	342,023 / 838	0.03% (0.25)
	fneg <8xdbl> ★	D4 / D11	8	8 / 1	35.36% (43.75)	15,547 / 1	280 / 1	13.42% (49.10)
	fma <8xdbl> ★	D4 / D11	8	8 / 4	70.71% (25.00)	313.97M / 261,973	40,021 / 810	0.06% (0.15)
Memory	load	D2 / D3 / D12	4,481	1 / 66 / 2	0.11% (0.68)	- / 4,064 / 1	- / 29 / 1	8.45% (49.64)
	store	D2 / D3 / D12 / D13	3,015	1 / 19 / 1 / 1	0.07% (0.26)	1 / 266 / 1 / 1	1 / 8 / 1 / 1	41.64% (42.00)

Table 6: PC Sensitivity and Input Breadth per opcode across all `CC-ITHICA` runs (opcodes with only one PC or one error are excluded). We mark with ★ opcodes for which all PCs belong to a single basic block.

	Top-level CC						Libraries					FB					
Test	**Avx**	**Hasher**	**Pattern_Gen**	**Malign_Buff**	**Silkscreen**	**Utils**	**Zlib**	**OpenSSL**	**Abseil**	**Llvmlibc**	**Llvmlibc++**	**Swissmap**	**Proto**	**Libc**	**Tcmalloc**	**Hashing**	**Stl-cord**
Total	19	3	21	17	6	4	23	9	7	0	3	4	8	3	9	11	7
Unique	15	0	7	3	3	0	4	1	0	0	0	1	3	0	3	1	3

Table 7: Total and unique server detections among CC, libraries and FB for DPool & QPool servers across all binaries.

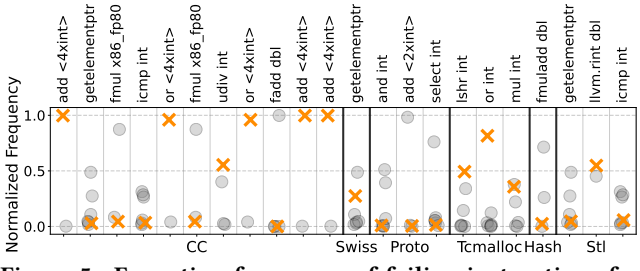


Figure 5: Execution frequency of failing instructions for servers (columns) uniquely detected by one ITHICA program. Orange indicates the detecting program; gray indicates non-detecting programs executing the same instruction.

Among `CC` subtests, Avx detects the most unique servers. This is expected since it is the only test exercising large (4x, 8x) vector operations. Among the instrumented libraries, Zlib detects the most servers (23 total). **Obs. 14** Notably, of the servers missed by `Native` in `CC-ITHICA` (Fig. 2), **eight are detected exclusively within library code**: six within Zlib alone, one within OpenSSL, and one shared between Zlib and OpenSSL. This demonstrates that even if `Native` checks were manually constructed to be more fine-grained at the top-level code, they would still miss errors that manifest within library code and are masked before reaching the library function's output. **Obs. 15** `FB` tests detect a total of 24 servers, including **11 unique servers** missed by both `CC` and libraries (not included in Fig. 2, for a total of 100 servers detected by ITHICA). Importantly, none of the `FB` programs could be used as tests without ITHICA, as they lack checkable outputs, underscoring the value of in-situ testing with arbitrary programs.

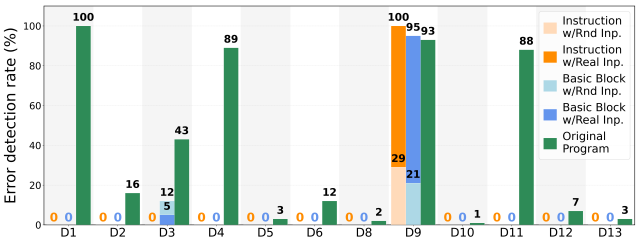
That different workloads detect different servers raises the question of what drives this program-sensitivity. Recent work emphasizes instruction usage stress (execution frequency of vulnerable instructions) as a primary triggering condition [82, 83]. To investigate whether execution frequency determines detection, we focus on servers uniquely detected by a single workload in our experiments. We analyze those where at least one failing instruction also appears in other workloads. For each such server, we select the most frequently failing shared instruction. Fig. 5 shows the normalized execution frequency of this instruction across all workloads that execute it. **Obs. 16** In 13 of 22 cases (59%), the detecting workload is *not* the one with the highest execution frequency. This further supports that execution context, not merely instruction execution stress, influence defect manifestation.

> **Finding 7:** The program-sensitivity of defect detection is not explained by execution frequency of the failing opcode; it is driven by the execution context that each program's instruction sequences create around the failing instruction (Obs. 14, 15, 16).

Implications for Chip-Users: *Instruction stress, emphasized by prior studies as a primary error trigger [82, 83], is not a reliable predictor of detection. Instead, what drives detection is the execution context that a program's instruction sequences create around the failing instruction. Since different programs create different contexts, using diverse programs as tests, as enabled by ITHICA, is essential. This provides an additional lever beyond those identified in §7.4.*

7.5 Reproducibility is Challenging

As shown in Fig. 1, not all runs of the same `CC-ITHICA` test result in detections for the same server. This can be attributed to *varying execution context* or *input dependence*. In the former case, microarchitectural (including architectural) and electrical state vary across



	D3		D9	
	Instr.	BB	Instr.	BB
Failing Opcode	None	or <4xint> / trunc int / zext int	fmul fp80	fmul fp80
Fail BB Length	—	17 / 13.5/ 25.0	—	4.0
Pass BB Length	—	12.0 / 1.0/ 16.2	—	—

Figure 6: *Top*: EDR of original CC-ITHICA program compared to single-instruction and single-basic-block reproducers (one per failing instruction, averaged). Reproducers are evaluated under both original failing inputs (light bars) and random inputs (dark bars). *Bottom*: For each reproducer, the failing opcodes and lengths of failing vs. non-failing basic blocks.

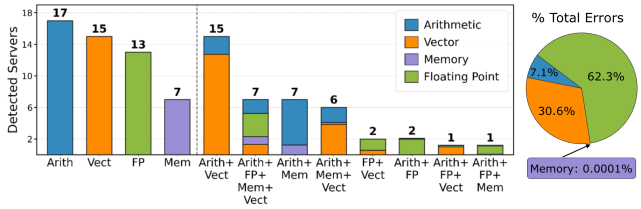


Figure 7: Combinations of failing instruction types across ITHICA tests. Color segments reflect average per-server error breakdown, along with overall error breakdown (pie chart).

runs of the same program on the same hardware [53]; thus, defect-induced errors manifest in some runs but not others. In the latter case, different runs use different randomly generated inputs, which influence manifestation (since instructions' inputs are part of their execution context) and detection (e.g., through logical masking), or even control flow (determining which instructions execute at all).

To isolate the effect of *non-input context* on the reproducibility of detections across runs, we rerun CC-ITHICA on three machines with high EDR (D3, D4, D7) with fixed inputs across runs, using TTD (Table 2) as a proxy for cross-run error reproducibility. Inputs were fixed randomly, as the high-level program inputs that produce errors are not directly accessible. If the same input repeatedly triggers the same error, TTD should remain constant.

Obs. 17 TTD distributions remain highly variable (e.g., for D3, ranging from seconds to nearly the full hour), demonstrating that non-input/-architectural execution context contributes to error manifestation. Other servers did not exhibit any error detections in this experiment, precluding further analysis.

To explore whether short tests derived from longer ITHICA tests can reproduce errors, we construct two additional test types per server: single-instruction tests containing only a failing instruction, and single-basic-block tests containing a failing basic block. For each, we evaluate both real failing inputs (from full test executions) and newly generated random inputs. We use Arith for all servers except D2, where we use Mem, and D12, where we use MemDiv, since they are uniquely detected by them. We run each experiment for 100 hours and report the results in Fig. 6, alongside the full-program rates from Fig. 1, and details for tests with detections at the bottom.

Obs. 18 Typically, neither single instructions nor basic blocks detect any errors, even with real failing inputs. There are two exceptions; D3 is not reproduced by single-instruction tests but is by basic-block tests, suggesting that smaller sequences establish the necessary execution context for its detection. D9, which fails on *fmul x86_fp80*, achieves perfect reproduction with the single-instruction tests with real inputs.

A potential explanation for D9 is reduced microarchitectural path non-determinism of this instruction: based on the executed assembly instruction, uops.info [2], and the server's microarchitecture documentation (undisclosed), this instruction maps to a single micro-op executed on one of two dedicated FPU units. The frequency of both original and validation instructions producing incorrect, inconsistent outputs suggests they often execute on the same defective FPU, assuming it is unlikely that both FPUs are concurrently defective. Notably, even with this reduced non-determinism, ITHICA still detects the error, making this case similar to our fault injection example in §3.

This experiment, along with the TTD variability observed under fixed program inputs (Obs. 17), indicates that microarchitectural and electrical state (beyond architectural control, as discussed in §3) contribute to the manifestation of defect-induced errors.

Finding 8: Reproduction of defect-induced errors generally cannot be done deterministically with ISA-level control (Obs.17, 18).

Having established that supplying the right inputs to the right opcodes is insufficient for detection, we further characterize the distribution of input values that trigger errors for each opcode-server combination for our full ITHICA programs, using **Input Breadth** (Table 2).

Obs. 19 Table 6 shows that, for most opcodes and servers, the number of unique failing inputs and Input Breadth are high, indicating that errors are largely not confined to specific corner-case input values.

Finding 9: For vulnerable instructions, selecting the "perfect" input appears to be neither necessary nor sufficient to induce a defect-induced error (Obs.17, 18, 19).

Implications for Chip-Users: *The reliance of prior approaches [15, 16, 51] on short tests or reproducers for error characterization, might bias their conclusions towards defects easily detectable by short sequences. Identification of vulnerable instructions must instead be performed concurrently with detection, across long, diverse programs.*

7.6 ISA-Level Hardware Localization is Difficult

Fig. 4 shows that some servers exhibit errors concentrated in a single instruction type (e.g., D10), while others show errors across types (e.g., D1, D12). Fig. 7 quantifies this across 93 of the total 100 ITHICA-detected servers (89 by CC-ITHICA and 11 unique to FB-ITHICA); seven are excluded from this analysis due to a crash that prevented logging of the failing instruction. For each combination of instruction types, it shows the number of servers exhibiting errors for that combination and the average per-server breakdown

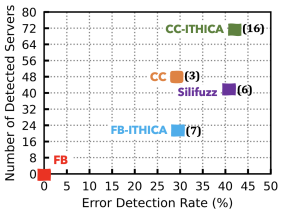


Figure 8: Comparison of SiliFuzz, CC and ITHICA tests, across all tested servers. Unique server detections for each test are shown in parenthesis.

of number of errors by instruction type; the accompanying pie chart aggregates this breakdown across all detected servers.

Obs. 20 44% of detected servers exhibit errors spanning multiple instruction types, rather than a single type—consistent with prior work observing multi-family test failures [12]; here, we observe this at the instruction granularity and quantify the rate disparity between types. **Obs. 21** Despite accounting for a large share of detected servers, arithmetic and memory instructions contribute only a small fraction of errors: 92.9% of all errors originate from floating-point and vector instructions. **Obs. 22** D1 illustrates the diagnostic consequences of this: the AVX subtest of CC (a vector-specialized test) is the only baseline test detecting D1. Yet ITHICA's instruction-level checks within the same test reveal that both vector and non-vector instructions are affected, pointing away from a vector-specific unit—a conclusion that would be missed without instruction-level visibility, leaving the defect misattributed based solely on test specialization.

Finding 10: The same defect often causes errors across multiple instruction types, at marginally different rates (Obs.20, 21, 22).

Implications for Chip-Users: *Hardware localization—attributing a defect to hardware components—from ISA-level observations is inherently difficult, if not impossible. Current techniques without instruction-level visibility across instruction types compound this difficulty. Therefore, conclusions drawn in prior works regarding vulnerable features and execution units [51, 82, 83] might reflect a detection artifact, guiding towards software and hardware mitigations targeting the wrong or insufficient components. Instruction-level checking across diverse instructions and programs is essential to avoid this.*

8 Discussion

Comparison with Fuzzing Techniques. Fig. 8 compares SiliFuzz [24, 37] (the only fuzzing tool operating at ISA-level or above), `CC`, `CC-ITHICA`, and `FB-ITHICA` by number of total and unique detected servers and average `EDR`; TTD and `EF` statistics are not available for SiliFuzz. This analysis is limited to servers that have accumulated at least 20 hours of execution for all tests. SiliFuzz detects significantly fewer servers than `CC-ITHICA` (42 vs. 71). For the servers it detects, it achieves slightly worse average error detection rate than `CC-ITHICA`. Notably, `FB-ITHICA` detects seven servers missed by all other tests.

9 Related Work

Soft Error Detection and Post Silicon Validation. EDDI [59] introduces intra-thread instruction checks first. SWIFT [69] reduces EDDI's overhead through optimized resource usage, and improves control-flow coverage. Several variations propose further performance optimizations [14, 19, 28, 32]. The QED family of techniques [31, 42, 43, 72] targets design bugs in post-silicon validation, adopting intra-thread checks from soft error techniques, as well as cross-core instruction-level checks. ITHICA adapts and re-implements for LLVM intra-thread instruction checks of these works, and extends them with the novel `MemDiv` transformation that addresses their limited memory coverage. `MemDiv` can benefit post-silicon validation but is inapplicable for soft errors.

		Alibaba [82, 83]	**SEVI** [51]	**ITHICA** *(this work)*
Evaluation	Suspect-Defect. Pool Size	Unclear	>2,500	>3,000
	Servers Detected & Analyzed	27-30	18	**100**
Testing Method	Error Assumption	Consistent (implicit)	Consistent (explicit)	Inconsistent (validated on HW, §7.1)
	Inst. Types Checked	— (not Inst.-level)	Vector only	All (Arith, FP, Vec, Mem, CF)
	Error Check Technique	Final Output w/ Golden Value	Inst.-level w/ Golden Value	Intra-thread Inst.-level
	Instruction Localization	Post-detection w/ Short Reproducers	Concurrent	Concurrent
Findings	Error Triggers	Opcode Stress, Temperature	Opcode, Input, Temperature	Sequence-Dependence; Opcode stress & inputs alone insufficient (§7.3-7.4)
	Vulnerable HW Unit	ALU, FPU, Vec.Unit Cache, TrxMem	Vector Multiplier	Prior claims likely unreliable (§7.6)
	Solution Proposed	Can focus on vulnerable features	ABFT for matmul kernels	Diverse programs as tests; Inst. localization concur. w/ detection (§7.5–7.6)

Table 8: Comparison with related hyperscalar fleet studies.

Hyperscalar Fleet Studies. Like ITHICA, the Alibaba studies [82, 83] and SEVI [51] evaluate their techniques using pools of suspect-defective servers. Table 8 compares their testing methods and findings with ITHICA's. Other works quantify the prevalence and report detection trends of hardware errors across large CPU fleets [12, 15–17, 30, 53] or study SDC vulnerability through fault injection [8, 22, 22, 64, 65]. Among them, PinDrop [12] advocates for continuous testing with a variety of complex tests at scale—a philosophy that aligns with ITHICA's goal of enabling arbitrary programs to be used as tests. Mitra et al. [53] finds that a significant fraction of defects are detected by production workloads in field, motivating the instrumentation of such workloads with ITHICA.

10 Conclusion

We present ITHICA, an approach for automatically generating functional tests from arbitrary programs to detect defective CPUs. ITHICA exploits the insight that the most pernicious defects manifest as inconsistent errors—validated here for the first time on real hardware—to build intra-thread, instruction-level checks that outperform baseline industry checks and enable novel findings that challenge conclusions of prior hyperscaler studies.

References

[1] Andreas Abel, Yuying Li, Richard O'Grady, Chris Kennelly, and Darryl Gove. 2024. A Profiling-Based Benchmark Suite for Warehouse-Scale Computers. In *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 325–327. https://doi.org/10.1109/ISPASS61541.2024.00046

[2] Andreas Abel and Jan Reineke. 2019. uops.info: Characterizing Latency, Throughput, and Port Usage of Instructions on Intel Microarchitectures. In *ASPLOS*

(Providence, RI, USA) *(ASPLOS '19)*. ACM, New York, NY, USA, 673–686. https://doi.org/10.1145/3297858.3304062

[3] abseil 2024. Abseil. https://github.com/abseil/abseil-cpp.

[4] Mridul Agarwal, Bipul C. Paul, Ming Zhang, and Subhasish Mitra. 2007. Circuit Failure Prediction and Its Application to Transistor Aging. In *25th IEEE VLSI Test Symposium (VTS'07)*. 277–286. https://doi.org/10.1109/VTS.2007.22

[5] Anonymous Authors. 2026. ITHICA. https://anonymous.4open.science/r/ithica-sdc.

[6] Jonathan T.-Y. Chang, Chao-Wen Tseng, Chien-Mo James Li, Mike Purtell, and Edward Joseph McCluskey. 1998. Analysis of pattern-dependent and timing-dependent failures in an experimental test chip. *Proceedings International Test Conference 1998 (IEEE Cat. No.98CH36270)* (1998), 184–193. https://api.semanticscholar.org/CorpusID:16286356

[7] Saranyu Chattopadhyay, Keerthikumara Devarajegowda, Bihan Zhao, Florian Lonsing, Brandon A. D'Agostino, Ioanna Vavelidou, Vijay D. Bhatt, Sebastian Prebeck, Wolfgang Ecker, Caroline Trippel, Clark Barrett, and Subhasish Mitra. 2023. G-QED: Generalized QED Pre-silicon Verification beyond Non-Interfering Hardware Accelerators. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. https://doi.org/10.1109/DAC56929.2023.10247903

[8] Odysseas Chatzopoulos, Nikos Karystinos, George Papadimitriou, Dimitris Gizopoulos, Harish D. Dixit, and Sriram Sankar. 2025. Veritas - Demystifying Silent Data Corruptions: uArch-Level Modeling and Fleet Data of Modern x86 CPUs. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1–14. https://doi.org/10.1109/HPCA61900.2025.00012

[9] Odysseas Chatzopoulos, George Papadimitriou, Dimitris Gizopoulos, Harish D Dixit, and Sriram Sankar. 2025. From gates to sdcs: Understanding fault propagation through the compute stack. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7.

[10] Tze Wee Chen, Kyunglok Kim, Young Moon Kim, and Subhasish Mitra. 2008. Gate-Oxide Early Life Failure Prediction. In *26th IEEE VLSI Test Symposium (vts 2008)*. 111–118. https://doi.org/10.1109/VTS.2008.55

[11] Eric Cheng, Shahrzad Mirkhani, Lukasz G. Szafaryn, Chen-Yong Cher, Hyungmin Cho, Kevin Skadron, Mircea R. Stan, Klas Lilja, Jacob A. Abraham, Pradip Bose, and Subhasish Mitra. 2016. CLEAR: Cross-Layer Exploration for Architecting Resilience - Combining hardware and software techniques to tolerate soft errors in processor cores. In *Proceedings of the 53rd Annual Design Automation Conference* (Austin, Texas) *(DAC '16)*. Association for Computing Machinery, New York, NY, USA, Article 68, 6 pages. https://doi.org/10.1145/2897937.2897996

[12] Peter Deutsch, Harish Dixit, Gautham Vunnam, Carl Moran, Eleanor Ozer, and Sriram Sankar. 2026. PinDrop: Breaking the Silence on SDCs in a Large-Scale Fleet. 1–14. https://doi.org/10.1109/HPCA68181.2026.11408620

[13] Peter W. Deutsch, Vincent Quentin Ulitzsch, Sudhanva Gurumurthi, Vilas Sridharan, Joel S. Emer, and Mengjia Yan. 2024. DelayAVF: Calculating Architectural Vulnerability Factors for Delay Faults. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 231–245. https://doi.org/10.1109/MICRO61859.2024.00026

[14] Moslem Didehban and Aviral Shrivastava. 2016. nZDC: A compiler technique for near Zero Silent Data Corruption. In *2016 53nd ACM/EDAC/IEEE Design Automation Conference (DAC)*. 1–6. https://doi.org/10.1145/2897937.2898054

[15] Harish Dattatraya Dixit, Laura Boyle, Gautham Vunnam, Sneha Pendharkar, Matt Beadon, and Sriram Sankar. 2022. Detecting silent data corruptions in the wild. arXiv:2203.08989 [cs.AR] https://arxiv.org/abs/2203.08989

[16] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. 2021. Silent Data Corruptions at Scale. *CoRR* abs/2102.11245 (2021). https://arxiv.org/abs/2102.11245.

[17] Rhea Dutta, Harish Dattatraya Dixit, Rik Van Riel, Gautham Vunnam, and Sriram Sankar. 2025. *Hardware Sentinel: Protecting Software Applications from Hardware Silent Data Corruptions*. Association for Computing Machinery, New York, NY, USA, 482–497. https://doi.org/10.1145/3676641.3716258

[18] E. B. Eichelberger and T. W. Williams. 1988. A logic design structure for LSI testability. In *Papers on Twenty-Five Years of Electronic Design Automation (25 years of DAC)*. Association for Computing Machinery, New York, NY, USA, 358–364. https://doi.org/10.1145/62882.62924

[19] Shuguang Feng, Shantanu Gupta, Amin Ansari, and Scott Mahlke. 2010. Shoestring: probabilistic soft error reliability on the cheap. *ACM SIGPLAN Notices* 45 (03 2010), 385. https://doi.org/10.1145/1735971.1736063

[20] Nikos Foutris, Dimitris Gizopoulos, Mihalis Psarakis, Xavier Vera, and Antonio Gonzalez. 2011. Accelerating microprocessor silicon validation by exposing ISA diversity. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture* (Porto Alegre, Brazil) *(MICRO-44)*. Association for Computing Machinery, New York, NY, USA, 386–397. https://doi.org/10.1145/2155620.2155666

[21] Nishant George, Sudhanva Gurumurthi, Vilas Sridharan, Harish Dattatraya Dixit, Emel Goksu, Bharath Parthasarathy, Amber Huffman, Thiago Macieira, Arani Sinha, Dean Liberty, Lisa Minwell, and Robert S. Chappell. 2025. Silent Data Corruption in AI: A Growing Challenge for Large-Scale Machine Learning. *IEEE Micro* (2025), 1–7. https://doi.org/10.1109/MM.2025.3645670

[22] Dimitris Gizopoulos, George Papadimitriou, Odysseas Chatzopoulos, Nikos Karystinos, Harish D. Dixit, and Sriram Sankar. 2024. Silent Data Corruptions in Computing Systems: Early Predictions and Large-Scale Measurements. In *2024 IEEE European Test Symposium (ETS)*. 1–10. https://doi.org/10.1109/ETS61313.2024.10567770

[23] Google. 2020. Google cpu-check torture test. https://github.com/google/cpu-check.

[24] Google. 2021. Silifuzz. https://github.com/google/silifuzz.

[25] Google. 2022. Fleetbench. https://github.com/google/fleetbench.

[26] F. Hapke, R. Krenz-Baath, A. Glowatz, J. Schloeffel, H. Hashempour, S. Eichenberger, C. Hora, and D. Adolfsson. 2009. Defect-oriented cell-aware ATPG and fault simulation for industrial cell libraries and designs. In *2009 International Test Conference*.

[27] Zhengyang He, Yafan Huang, Hui Xu, Dingwen Tao, and Guanpeng Li. 2023. Demystifying and Mitigating Cross-Layer Deficiencies of Soft Error Protection in Instruction Duplication. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, CO, USA) *(SC '23)*. Association for Computing Machinery, New York, NY, USA, Article 86, 13 pages. https://doi.org/10.1145/3581784.3607078

[28] Zhengyang He, Hui Xu, and Guanpeng Li. 2024. A Fast Low-Level Error Detection Technique. In *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 90–98. https://doi.org/10.1109/DSN58291.2024.00023

[29] K. Heragu, J.H. Patel, and V.D. Agrawal. 1996. Segment delay faults: a new fault model. In *Proceedings of 14th VLSI Test Symposium*.

[30] Peter H. Hochschild, Paul Turner, Jeffrey C. Mogul, Rama Govindaraju, Parthasarathy Ranganathan, David E. Culler, and Amin Vahdat. 2021. Cores That Don't Count. In *Proceedings of the Workshop on Hot Topics in Operating Systems*.

[31] Ted Hong, Yanjing Li, Sung-Boem Park, Diana Mui, David Lin, Ziyad Abdel Kaleq, Nagib Hakim, Helia Naeimi, Donald S. Gardner, and Subhasish Mitra. 2010. QED: Quick Error Detection tests for effective post-silicon validation. In *2010 IEEE International Test Conference*.

[32] Yafan Huang, Shengjian Guo, Sheng Di, Guanpeng Li, and Franck Cappello. 2022. Mitigating Silent Data Corruptions in HPC Applications across Multiple Program Inputs. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14. https://doi.org/10.1109/SC41404.2022.00022

[33] Intel. 2021. OpenDCDiag. https://github.com/opendcdiag.

[34] Nikos Karystinos, Odysseas Chatzopoulos, George-Marios Fragkoulis, George Papadimitriou, Dimitris Gizopoulos, and Sudhanva Gurumurthi. 2024. Harpocrates: Breaking the Silence of CPU Faults through Hardware-in-the-Loop Program Generation. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 516–531. https://doi.org/10.1109/ISCA59077.2024.00045

[35] Nikos Karystinos, George-Marios Fragkoulis, Odysseas Chatzopoulos, Dimitris Gizopoulos, and Sudhanva Gurumurthi. 2025. Harpocrates++: Automated Functional Program Generation against CPU Faults and Silent Data Corruptions. *IEEE Micro* (2025), 1–9. https://doi.org/10.1109/MM.2025.3640385

[36] S. Kundu, S. Sengupta, and R. Galivanche. 2000. Test challenges in nanometer technologies. In *Proceedings IEEE European Test Workshop*.

[37] Doug Kwan, Kostik Shtoyk, Kostya Serebryany, Maxim L Lifantsev, and Peter Hochschild. 2021. *SiliFuzz: Fuzzing CPUs by proxy*. Technical Report. Google.

[38] J.C.-M. Li and E.J. McCluskey. 2002. Diagnosis of sequence-dependent chips. In *Proceedings 20th IEEE VLSI Test Symposium (VTS 2002)*. 187–192. https://doi.org/10.1109/VTS.2002.1011137

[39] Wei Li, Chris Nigh, Danielle Duvalsaint, Subhasish Mitra, and R. D. Blanton. 2022. PEPR: Pseudo-Exhaustive Physically-Aware Region Testing. In *International Test Conference*.

[40] libcllvm 2024. The LLVM C Library. https://libc.llvm.org/.

[41] libcxxllvm 2024. LLVM libc++. https://github.com/llvm/llvm-project/blob/main/libcxx/include/concepts.

[42] David Lin, Ted Hong, Yanjing Li, Farzan Fallah, Donald S Gardner, Nagib Hakim, and Subhasish Mitra. 2013. Overcoming post-silicon validation challenges through quick error detection (QED). In *2013 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 320–325.

[43] David Lin, Ted Hong, Yanjing Li, Eswaran S, Sharad Kumar, Farzan Fallah, Nagib Hakim, Donald S. Gardner, and Subhasish Mitra. 2014. Effective Post-Silicon Validation of System-on-Chips Using Quick Error Detection. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 33, 10 (2014), 1573–1590. https://doi.org/10.1109/TCAD.2014.2334301

[44] llvm-language-ref 2022. LLVM Language Reference Manual. https://llvm.org/docs/LangRef.html. Accessed: 2022-10-19.

[45] Florian Lonsing, Subhasish Mitra, and Clark W. Barrett. 2020. A Theoretical Framework for Symbolic Quick Error Detection. In *2020 Formal Methods in Computer Aided Design, FMCAD 2020, Haifa, Israel, September 21-24, 2020*. IEEE, 1–10. https://doi.org/10.34727/2020/ISBN.978-3-85448-042-6_9

[46] Jiacheng Ma, Majd Ganaiem, Madeline Burbage, Theo Gregersen, Rachel McAmis, Freddy Gabbay, and Baris Kasikci. 2025. Proactive Runtime Detection of Aging-Related Silent Data Corruptions: A Bottom-Up Approach. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming*

Languages and Operating Systems, Volume 4 (Hilton La Jolla Torrey Pines, La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 220–235. https://doi.org/10.1145/3622781.3674182

[47] S.C. Ma, P. Franco, and E.J. McCluskey. 1995. An experimental chip to evaluate test techniques experiment results. In *Proceedings of 1995 IEEE International Test Conference (ITC)*.

[48] Timothy C. May and Murray H. Woods. 1978. A New Physical Mechanism for Soft Errors in Dynamic Memories. In *16th International Reliability Physics Symposium*. 33–40. https://doi.org/10.1109/IRPS.1978.362815

[49] E.J. McCluskey. 1993. Quality and single-stuck faults. In *Proceedings of IEEE International Test Conference - (ITC)*.

[50] E.J. McCluskey and Chao-Wen Tseng. 2000. Stuck-fault tests vs. actual defects. In *Proceedings International Test Conference 2000 (IEEE Cat. No.00CH37159)*.

[51] Yixuan Mei, Shreya Varshini, Harish Dixit, Sriram Sankar, and K. V. Rashmi. 2026. SEVI: Silent Data Corruption of Vector Instructions in Hyper-Scale Datacenters. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (USA) *(ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1711–1726. https://doi.org/10.1145/3779212.3790217

[52] Justin Meza, Qiang Wu, Sanjeev Kumar, and Onur Mutlu. 2015. Revisiting Memory Errors in Large-Scale Production Data Centers: Analysis and Modeling of New Trends from the Field. In *2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. 415–426. https://doi.org/10.1109/DSN.2015.57

[53] Subhasish Mitra, Subho Banerjee, Martin Dixon, Rama Govindaraju, Peter Hochschild, Eric X. Liu, Bharath Parthasarathy, and Parthasarathy Ranganathan. 2025. Silent Data Corruption by 10x Test Escapes Threatens Reliable Computing. arXiv:2508.01786 [cs.AR] https://arxiv.org/abs/2508.01786

[54] S. Mukherjee. 2008. *Architecture Design for Soft Errors*. https://doi.org/10.1016/B978-0-12-369529-1.X5001-0

[55] S.S. Mukherjee, J. Emer, and S.K. Reinhardt. 2005. The soft error problem: an architectural perspective. In *11th International Symposium on High-Performance Computer Architecture*.

[56] S.S. Mukherjee, C. Weaver, J. Emer, S.K. Reinhardt, and T. Austin. 2003. A systematic methodology to compute the architectural vulnerability factors for a high-performance microprocessor. In *Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture, 2003. MICRO-36*. 29–40. https://doi.org/10.1109/MICRO.2003.1253181

[57] N. Oh, S. Mitra, and E.J. McCluskey. 2002. ED4I: error detection by diverse data and duplicated instructions. *IEEE Trans. Comput.* 51, 2 (2002), 180–199. https://doi.org/10.1109/12.980007

[58] N. Oh, P.P. Shirvani, and E.J. McCluskey. 2002. Control-flow checking by software signatures. *IEEE Transactions on Reliability* 51, 1 (2002), 111–122. https://doi.org/10.1109/24.994926

[59] Nahmsuk Oh, Philip Shirvani, and Edward McCluskey. 2002. Error detection by duplicated instructions in super-scalar processors. *IEEE Transactions on Reliability* 51, 1 (2002), 63–75. https://doi.org/10.1109/24.994913

[60] OpenHW Group. 2019. CVA6 RISC-V CPU. https://github.com/openhwgroup/cva6.

[61] openssl 2024. OpenSSL. https://github.com/openssl/openssl.

[62] openssl-manual [n. d.]. OPENSSL Debian Manpages. https://manpages.debian.org/testing/libssl-doc/OPENSSL_LH_doall_arg.3ssl.en.html.

[63] George Papadimitriou and Dimitris Gizopoulos. 2023. AVGI: Microarchitecture-Driven, Fast and Accurate Vulnerability Assessment. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 935–948. https://doi.org/10.1109/HPCA56546.2023.10071105

[64] George Papadimitriou and Dimitris Gizopoulos. 2023. Silent Data Corruptions: Microarchitectural Perspectives. *IEEE Trans. Comput.* 72, 11 (2023), 3072–3085. https://doi.org/10.1109/TC.2023.3285094

[65] George Papadimitriou, Dimitris Gizopoulos, Harish Dattatraya Dixit, and Sriram Sankar. 2023. Silent Data Corruptions: The Stealthy Saboteurs of Digital Integrity. *2023 IEEE 29th International Symposium on On-Line Testing and Robust System Design (IOLTS)* (2023), 1–7. https://api.semanticscholar.org/CorpusID:261315246

[66] Priyadarsan Patra. 2007. On the cusp of a validation wall. *IEEE Design & Test of Computers* 24, 2 (2007), 193–196. https://doi.org/10.1109/MDT.2007.54

[67] Bipul C. Paul, Kunhyuk Kang, Haldun Kufluoglu, Muhammad A. Alam, and Kaushik Roy. 2007. Negative Bias Temperature Instability: Estimation and Design for Improved Reliability of Nanoscale Circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 26, 4 (2007), 743–751. https://doi.org/10.1109/TCAD.2006.884870

[68] Mahesh Prabhu and Jacob A. Abraham. 2012. Functional test generation for hard to detect stuck-at faults using RTL model checking. In *2012 17th IEEE European Test Symposium (ETS)*.

[69] G.A. Reis, J. Chang, N. Vachharajani, R. Rangan, and D.I. August. 2005. SWIFT: software implemented fault tolerance. In *International Symposium on Code Generation and Optimization*. 243–254. https://doi.org/10.1109/CGO.2005.34

[70] Matthias Sauer, Young Moon Kim, Jun Seomun, Hyung-Ock Kim, Kyung-Tae Do, Jung Yun Choi, Kee Sup Kim, Subhasish Mitra, and Bernd Becker. 2013. Early-life-failure detection using SAT-based ATPG. In *2013 IEEE International Test Conference (ITC)*. 1–10. https://doi.org/10.1109/TEST.2013.6651925

[71] Jian Shen and Jacob A. Abraham. 1998. Native mode functional test generation for processors with applications to self test and design validation. *Proceedings International Test Conference 1998 (IEEE Cat. No.98CH36270)* (1998), 990–999. https://api.semanticscholar.org/CorpusID:14132281

[72] Eshan Singh, Clark W. Barrett, and Subhasish Mitra. 2017. E-QED: Electrical Bug Localization During Post-silicon Validation Enabled by Quick Error Detection and Formal Methods. (2017).

[73] Gordon L. Smith. 1985. Model for Delay Faults Based upon Paths. In *International Test Conference*.

[74] Wilson Snyder, Paul Wasson, Duane Galbi, and et al. [n. d.]. *Verilator*. https://verilator.org

[75] Daniel Sorin. 2009. *Fault Tolerant Computer Architecture*. Vol. 4. https://doi.org/10.2200/S00192ED1V01Y200904CAC005

[76] T.M. Storey and W. Maly. 1990. CMOS bridging fault detection. In *Proceedings. International Test Conference 1990*.

[77] E. Takeda and N. Suzuki. 1983. An empirical model for device degradation due to hot-carrier injection. *IEEE Electron Device Letters* 4, 4 (1983), 111–113. https://doi.org/10.1109/EDL.1983.25667

[78] Arman Vassighi, Richard Kacprowicz, Carlos Carranza, and Walter Riordan. 2008. Characterizing infant mortality in high volume manufacturing. In *2008 IEEE International Reliability Physics Symposium*. 717–718. https://doi.org/10.1109/RELPHY.2008.4559004

[79] R. L. Wadsack. 1978. Fault modeling and logic simulation of CMOS and MOS integrated circuits. *The Bell System Technical Journal* 57, 5 (1978), 1449–1474. https://doi.org/10.1002/j.1538-7305.1978.tb02106.x

[80] Ilya Wagner and Valeria Bertacco. 2008. Reversi: Post-silicon validation system for modern microprocessors. In *2008 IEEE International Conference on Computer Design*. 307–314. https://doi.org/10.1109/ICCD.2008.4751878

[81] John A. Waicukauski, Eric Lindbloom, Barry K. Rosen, and Vijay S. Iyengar. 1987. Transition Fault Simulation. *IEEE Design & Test of Computers* 4, 2 (1987), 32–38. https://doi.org/10.1109/MDT.1987.295104

[82] Shaobu Wang, Guangyan Zhang, Junyu Wei, Yang Wang, Jiesheng Wu, and Qingchao Luo. 2023. Understanding Silent Data Corruptions in a Large Production CPU Population. In *Proceedings of the 29th Symposium on Operating Systems Principles*.

[83] Shaobu Wang, Guangyan Zhang, Junyu Wei, Yang Wang, Jiesheng Wu, and Qingchao Luo. 2024. Understanding Silent Data Corruption in Processors for Mitigating its Effects. *ACM Trans. Archit. Code Optim.* 21, 4, Article 84 (Nov. 2024), 27 pages. https://doi.org/10.1145/3690825

[84] James F Ziegler and William A Lanford. 1979. Effect of cosmic rays on computer memories. *Science* 206, 4420 (1979), 776–788.

[85] zlib 2024. Zlib. https://zlib.net/.

11 Supplemental Material

11.1 ITHICA LLVM Implementation

We implement each of the four ITHICA transformations, as well as their combinations (`Arith+Mem`, `Arith+MemDiv`, `Arith+MemDiv+Br`) as distinct LLVM IR compiler passes. Each pass can be configured with different block size and interleaving values via command-line options. The resulting shared object file (`.so`) integrates directly into Clang-14's built-in compiler pipeline. Libraries managed by the build system are automatically compiled with our passes, while those explicitly linked are manually compiled with our passes (§5.1). Implementing ITHICA at the LLVM IR level provides portability, as IR passes are target-agnostic and can be integrated into existing build systems without requiring modifications to the compiler.

Correct instrumentation requires validation instructions to be side-effect free (to preserve program behavior) and that original and validation instructions yield the same output in the absence of errors. All LLVM instructions instrumented by `Arith` satisfy these properties, and can therefore be safely duplicated. For LLVM intrinsics and inline assembly, also instrumented by `Arith`, we verify via attributes that they are side-effect free before duplication. `Br` inserts a validation stack store and load with no side-effects.

For `Mem` and `MemDiv`, we exclude atomic/volatile loads and stores due to their memory side-effects. Other instructions that are excluded for the same reason are *alloca*, *atomicrmw*, and *cmpexchange* [44]; *fence* does not have an architectural effect to check. Additionally, even non-atomic/volatile loads and stores can exhibit data races in thread-unsafe multithreaded code. OpenSSL intentionally permits certain data races [62], so we do not instrument it with `Mem` and `MemDiv`. Despite these restrictions that limit coverage (i.e., fraction of checked instructions), `Arith` *and* `Br` *safely instrument all code*, and *thread-safe code is safely instrumented by all passes.*

Finally, ITHICA also excludes `void` instructions (except for stores) and structural instructions whose duplication would affect the correct SSA form of LLVM: *phi*, *landingpad*, *catchpad*, *cleanuppad*.

Table 10 shows the number of LLVM opcodes and static instructions (program counters, PCs) covered by each ITHICA pass. Combined, ITHICA passes instrument a total of 84% of opcodes and 84.8% of all static instructions in cpu-check (§5.1), among the ones with checkable architectural effects.

We apply ITHICA as the final IR-level pass, to minimize interference with compiler optimization passes. Despite that, backend compiler optimizations can still eliminate our inserted instructions. To prevent this without modifying compiler internals, we mark duplicated loads and stores as volatile. For arithmetic instructions, we pass their arguments through no-op volatile store-load pairs to achieve the same effect. Future versions of Clang may offer intrinsics to preserve side-effect-free instructions more cleanly.

ITHICA instrumentation can affect how the program's instructions are lowered to assembly. To ensure fair evaluation (§7), we primarily compare ITHICA checks against cpu-check's final output checks within the *same* ITHICA-compiled binary, rather than across independently compiled binaries (i.e., before and after ITHICA).

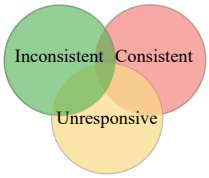


Figure 9: Classification of how hardware errors can manifest as three types of architectural errors [7, 45]. ITHICA explicitly detects pernicious inconsistent errors and implicitly detects unresponsive errors.

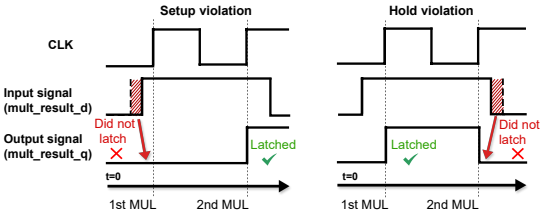


Figure 10: Physical fault model of single-bit setup and hold violations on the CVA6 multiplier output. In both cases, the two `MUL` instructions produce different results, allowing ITHICA to detect the fault.

Technique	Check Type				Evaluation		Spec
	Golden Value	Cross-Core	Invert. Comp.	Intra-thr. Instruct.	Fault Inject.	Real HW	
cpu-check [23]	✗	✓	✓	✗	–	–	C/C++
OpenDCDiag [33]	✓	✓	✓	✗	✗	✓	C/C++
SiliFuzz [37]	✓	✓	✗	✗	–	–	ASM
Harpocrates [34]	✓	✓	✗	✗	✓	✗	Uarch
ITHICA (this paper)	✗	✗	✗	✓	✗	✓	LLVM

Table 11: Testing tools for defects compared by check type, evaluation platform, and required specification level.

Industrial Functional Tests for Defects. OpenDCDiag [33] is a framework-based suite of compute-intensive tests (e.g., MatMul), validated against precomputed golden values or across cores, and inverse computations (e.g., encryption-decryption). Cpu-check [23] is discussed in §5.1. An orthogonal approach for test generation uses fuzzing to generate random instructions and expand short test cases [34, 35, 37], validating final outputs against golden values or across cores. Since defect manifestation appears sensitive to microarchitectural subtleties, the dependence of these approaches on CPU emulators and simulators may limit their effectiveness on real silicon. Reliance on below-ISA specifications for test generation [34, 35, 46] only accessible to chip manufacturers, renders these approaches inapplicable for chip users, including hyperscalars (§1)—ITHICA's focus. Nonetheless, ITHICA checks could be applied to fuzzer-generated sequences to provide automated error detection without relying on final outputs. We directly compare with Silifuzz, the only fuzzing technique publicly available (§8, Fig.8).

Coverage Gaps. We identify three main sources of limited coverage in our implementation: (1) atomic/volatile memory instructions (§11.1), (2) memory instructions in thread-unsafe code (§9) and (3) uninstrumented parts of `libc` and `libc++` libraries (§5.1). For (2), even though ITHICA does not cover memory instructions of thread-unsafe multithreaded programs, it still covers most application-level instructions for a class of programs that are difficult to check with other techniques (§1). For (3), by excluding the parts of `libc` and

Transform.	Target Instructions	Original Instr.	Inserted Validation Instructions	Inserted Check
`Arith`	Logic, binary, bitwise, vector, unary, aggregate & conversion ops, *getelementptr*, *icmp*, *fcmp* *select*, side-effect-free intrinsics & inline asm [44]	`r0 = op(s0,s1)`	`r1 = op(s0,s1)`	`eq(r0,r1)`
`Mem`	Load (non-atomic, non-volatile)	`v0 = load(a0)`	`v1 = load(a0)`	`eq(v0,v1)`
	Store (non-atomic, non-volatile)	`store(v0,a0)`	`v1 = load(a0)`	`eq(v0,v1)`
`MemDiv`	Load (non-atomic, non-volatile)	`v0 = load(a0)`	`v1 = load(a0); clflush(a0);` `v2 = load(a0)`	`eq(v0,v1) &&` `eq(v0,v2)`

	Store (non-atomic, non-volatile)	`store(v0,a0)`	`v1 = load(a0); mfence();`
			`v2 = load(a0); clflush(a0);`
			`v3 = load(a0)`
				`eq(v0,v1) &&`
				`eq(v0,v2) &&`
				`eq(v0, v3)`

`Br`	Conditional branches	`br(c0, t0,t1)`	`src: e = c0?t1:t0; store(e,tmp);`
			`t0: v0 = load(tmp);`
			`t1: v1 = load(tmp);`
				`src: None`
				`t0: eq(v0, t0)`
				`t1: eq(v1, t1)`

Table 9: Summary of ITHICA Transformations and Validation Rules

LLVM Instructions	`Arith`	`Mem` / `MemDiv`	`Br`	Excluded
Opcodes (# / %)	39 / 78.0%	2 / 4.0%	1 / 2.0%	8 / 16.0%
Static Instructions (PCs) (%)	62.3%	21.1%	1.4%	15.2%

Table 10: Coverage of different `ITHICA` passes for `CC` program.

`libc++` not supported by `llvmlibc` and `llvmlibc++`, we leave kernel code executed as part of system calls unchecked, though ITHICA targets application-level code.

Consistent Errors. Although it is not possible to determine whether the six machines missed by ITHICA (Fig. 2) are due to the above coverage gaps or consistent errors, future work can apply functional diversity [20, 42, 57, 80] to target the latter.

References

[1] Andreas Abel, Yuying Li, Richard O'Grady, Chris Kennelly, and Darryl Gove. 2024. A Profiling-Based Benchmark Suite for Warehouse-Scale Computers. In *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 325–327. https://doi.org/10.1109/ISPASS61541.2024.00046

[2] Andreas Abel and Jan Reineke. 2019. uops.info: Characterizing Latency, Throughput, and Port Usage of Instructions on Intel Microarchitectures. In *ASPLOS* (Providence, RI, USA) *(ASPLOS '19)*. ACM, New York, NY, USA, 673–686. https://doi.org/10.1145/3297858.3304062

[3] abseil 2024. Abseil. https://github.com/abseil/abseil-cpp.

[4] Mridul Agarwal, Bipul C. Paul, Ming Zhang, and Subhasish Mitra. 2007. Circuit Failure Prediction and Its Application to Transistor Aging. In *25th IEEE VLSI Test Symposium (VTS'07)*. 277–286. https://doi.org/10.1109/VTS.2007.22

[5] Anonymous Authors. 2026. ITHICA. https://anonymous.4open.science/r/ithica-sdc.

[6] Jonathan T.-Y. Chang, Chao-Wen Tseng, Chien-Mo James Li, Mike Purtell, and Edward Joseph McCluskey. 1998. Analysis of pattern-dependent and timing-dependent failures in an experimental test chip. *Proceedings International Test Conference 1998 (IEEE Cat. No.98CH36270)* (1998), 184–193. https://api.semanticscholar.org/CorpusID:16286356

[7] Saranyu Chattopadhyay, Keerthikumara Devarajegowda, Bihan Zhao, Florian Lonsing, Brandon A. D'Agostino, Ioanna Vavelidou, Vijay D. Bhatt, Sebastian Prebeck, Wolfgang Ecker, Caroline Trippel, Clark Barrett, and Subhasish Mitra. 2023. G-QED: Generalized QED Pre-silicon Verification beyond Non-Interfering Hardware Accelerators. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. https://doi.org/10.1109/DAC56929.2023.10247903

[8] Odysseas Chatzopoulos, Nikos Karystinos, George Papadimitriou, Dimitris Gizopoulos, Harish D. Dixit, and Sriram Sankar. 2025. Veritas - Demystifying Silent Data Corruptions: uArch-Level Modeling and Fleet Data of Modern x86 CPUs. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1–14. https://doi.org/10.1109/HPCA61900.2025.00012

[9] Odysseas Chatzopoulos, George Papadimitriou, Dimitris Gizopoulos, Harish D Dixit, and Sriram Sankar. 2025. From gates to sdcs: Understanding fault propagation through the compute stack. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7.

[10] Tze Wee Chen, Kyunglok Kim, Young Moon Kim, and Subhasish Mitra. 2008. Gate-Oxide Early Life Failure Prediction. In *26th IEEE VLSI Test Symposium (vts 2008)*. 111–118. https://doi.org/10.1109/VTS.2008.55

[11] Eric Cheng, Shahrzad Mirkhani, Lukasz G. Szafaryn, Chen-Yong Cher, Hyungmin Cho, Kevin Skadron, Mircea R. Stan, Klas Lilja, Jacob A. Abraham, Pradip Bose, and Subhasish Mitra. 2016. CLEAR: Cross-Layer Exploration for Architecting Resilience - Combining hardware and software techniques to tolerate soft errors in processor cores. In *Proceedings of the 53rd Annual Design Automation Conference* (Austin, Texas) *(DAC '16)*. Association for Computing Machinery, New York, NY, USA, Article 68, 6 pages. https://doi.org/10.1145/2897937.2897996

[12] Peter Deutsch, Harish Dixit, Gautham Vunnam, Carl Moran, Eleanor Ozer, and Sriram Sankar. 2026. PinDrop: Breaking the Silence on SDCs in a Large-Scale Fleet. 1–14. https://doi.org/10.1109/HPCA68181.2026.11408620

[13] Peter W. Deutsch, Vincent Quentin Ulitzsch, Sudhanva Gurumurthi, Vilas Sridharan, Joel S. Emer, and Mengjia Yan. 2024. DelayAVF: Calculating Architectural Vulnerability Factors for Delay Faults. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 231–245. https://doi.org/10.1109/MICRO61859.2024.00026

[14] Moslem Didehban and Aviral Shrivastava. 2016. nZDC: A compiler technique for near Zero Silent Data Corruption. In *2016 53nd ACM/EDAC/IEEE Design Automation Conference (DAC)*. 1–6. https://doi.org/10.1145/2897937.2898054

[15] Harish Dattatraya Dixit, Laura Boyle, Gautham Vunnam, Sneha Pendharkar, Matt Beadon, and Sriram Sankar. 2022. Detecting silent data corruptions in the wild. arXiv:2203.08989 [cs.AR] https://arxiv.org/abs/2203.08989

[16] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. 2021. Silent Data Corruptions at Scale. *CoRR* abs/2102.11245 (2021). https://arxiv.org/abs/2102.11245.

[17] Rhea Dutta, Harish Dattatraya Dixit, Rik Van Riel, Gautham Vunnam, and Sriram Sankar. 2025. *Hardware Sentinel: Protecting Software Applications from Hardware Silent Data Corruptions.* Association for Computing Machinery, New York, NY, USA, 482–497. https://doi.org/10.1145/3676641.3716258

[18] E. B. Eichelberger and T. W. Williams. 1988. A logic design structure for LSI testability. In *Papers on Twenty-Five Years of Electronic Design Automation (25 years of DAC)*. Association for Computing Machinery, New York, NY, USA, 358–364. https://doi.org/10.1145/62882.62924

[19] Shuguang Feng, Shantanu Gupta, Amin Ansari, and Scott Mahlke. 2010. Shoestring: probabilistic soft error reliability on the cheap. *ACM SIGPLAN Notices* 45 (03 2010), 385. https://doi.org/10.1145/1735971.1736063

[20] Nikos Foutris, Dimitris Gizopoulos, Mihalis Psarakis, Xavier Vera, and Antonio Gonzalez. 2011. Accelerating microprocessor silicon validation by exposing ISA diversity. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture* (Porto Alegre, Brazil) *(MICRO-44)*. Association for Computing Machinery, New York, NY, USA, 386–397. https://doi.org/10.1145/2155620.2155666

[21] Nishant George, Sudhanva Gurumurthi, Vilas Sridharan, Harish Dattatraya Dixit, Emel Goksu, Bharath Parthasarathy, Amber Huffman, Thiago Macieira, Arani Sinha, Dean Liberty, Lisa Minwell, and Robert S. Chappell. 2025. Silent Data Corruption in AI: A Growing Challenge for Large-Scale Machine Learning. *IEEE Micro* (2025), 1–7. https://doi.org/10.1109/MM.2025.3645670

[22] Dimitris Gizopoulos, George Papadimitriou, Odysseas Chatzopoulos, Nikos Karystinos, Harish D. Dixit, and Sriram Sankar. 2024. Silent Data Corruptions in Computing Systems: Early Predictions and Large-Scale Measurements. In *2024 IEEE European Test Symposium (ETS)*. 1–10. https://doi.org/10.1109/ETS61313.2024.10567770

[23] Google. 2020. Google cpu-check torture test. https://github.com/google/cpu-check.

[24] Google. 2021. Silifuzz. https://github.com/google/silifuzz.

[25] Google. 2022. Fleetbench. https://github.com/google/fleetbench.

[26] F. Hapke, R. Krenz-Baath, A. Glowatz, J. Schloeffel, H. Hashempour, S. Eichenberger, C. Hora, and D. Adolfsson. 2009. Defect-oriented cell-aware ATPG and

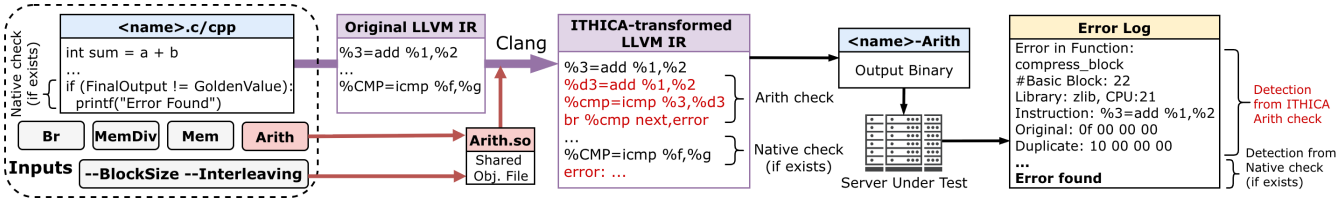


Figure 11: Given an input program (`<name>.cpp`), ITHICA applies one or more transformations (§4.1)—implemented in this paper for LLVM IR—configured with some `BlockSize` and `Interleaving` (§4.2), and outputs a functional test (`<name>-ITHICA`).

fault simulation for industrial cell libraries and designs. In *2009 International Test Conference*.

[27] Zhengyang He, Yafan Huang, Hui Xu, Dingwen Tao, and Guanpeng Li. 2023. Demystifying and Mitigating Cross-Layer Deficiencies of Soft Error Protection in Instruction Duplication. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, CO, USA) *(SC '23)*. Association for Computing Machinery, New York, NY, USA, Article 86, 13 pages. https://doi.org/10.1145/3581784.3607078

[28] Zhengyang He, Hui Xu, and Guanpeng Li. 2024. A Fast Low-Level Error Detection Technique. In *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 90–98. https://doi.org/10.1109/DSN58291.2024.00023

[29] K. Heragu, J.H. Patel, and V.D. Agrawal. 1996. Segment delay faults: a new fault model. In *Proceedings of 14th VLSI Test Symposium*.

[30] Peter H. Hochschild, Paul Turner, Jeffrey C. Mogul, Rama Govindaraju, Parthasarathy Ranganathan, David E. Culler, and Amin Vahdat. 2021. Cores That Don't Count. In *Proceedings of the Workshop on Hot Topics in Operating Systems*.

[31] Ted Hong, Yanjing Li, Sung-Boem Park, Diana Mui, David Lin, Ziyad Abdel Kaleq, Nagib Hakim, Helia Naeimi, Donald S. Gardner, and Subhasish Mitra. 2010. QED: Quick Error Detection tests for effective post-silicon validation. In *2010 IEEE International Test Conference*.

[32] Yafan Huang, Shengjian Guo, Sheng Di, Guanpeng Li, and Franck Cappello. 2022. Mitigating Silent Data Corruptions in HPC Applications across Multiple Program Inputs. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14. https://doi.org/10.1109/SC41404.2022.00022

[33] Intel. 2021. OpenDCDiag. https://github.com/opendcdiag.

[34] Nikos Karystinos, Odysseas Chatzopoulos, George-Marios Fragkoulis, George Papadimitriou, Dimitris Gizopoulos, and Sudhanva Gurumurthi. 2024. Harpocrates: Breaking the Silence of CPU Faults through Hardware-in-the-Loop Program Generation. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 516–531. https://doi.org/10.1109/ISCA59077.2024.00045

[35] Nikos Karystinos, George-Marios Fragkoulis, Odysseas Chatzopoulos, Dimitris Gizopoulos, and Sudhanva Gurumurthi. 2025. Harpocrates++: Automated Functional Program Generation against CPU Faults and Silent Data Corruptions. *IEEE Micro* (2025), 1–9. https://doi.org/10.1109/MM.2025.3640385

[36] S. Kundu, S. Sengupta, and R. Galivanche. 2000. Test challenges in nanometer technologies. In *Proceedings IEEE European Test Workshop*.

[37] Doug Kwan, Kostik Shtoyk, Kostya Serebryany, Maxim L Lifantsev, and Peter Hochschild. 2021. *SiliFuzz: Fuzzing CPUs by proxy*. Technical Report. Google.

[38] J.C.-M. Li and E.J. McCluskey. 2002. Diagnosis of sequence-dependent chips. In *Proceedings 20th IEEE VLSI Test Symposium (VTS 2002)*. 187–192. https://doi.org/10.1109/VTS.2002.1011137

[39] Wei Li, Chris Nigh, Danielle Duvalsaint, Subhasish Mitra, and R. D. Blanton. 2022. PEPR: Pseudo-Exhaustive Physically-Aware Region Testing. In *International Test Conference*.

[40] libcllvm 2024. The LLVM C Library. https://libc.llvm.org/.

[41] libcxxllvm 2024. LLVM libc++. https://github.com/llvm/llvm-project/blob/main/libcxx/include/concepts.

[42] David Lin, Ted Hong, Yanjing Li, Farzan Fallah, Donald S Gardner, Nagib Hakim, and Subhasish Mitra. 2013. Overcoming post-silicon validation challenges through quick error detection (QED). In *2013 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 320–325.

[43] David Lin, Ted Hong, Yanjing Li, Eswaran S, Sharad Kumar, Farzan Fallah, Nagib Hakim, Donald S. Gardner, and Subhasish Mitra. 2014. Effective Post-Silicon Validation of System-on-Chips Using Quick Error Detection. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 33, 10 (2014), 1573–1590. https://doi.org/10.1109/TCAD.2014.2334301

[44] llvm-language-ref 2022. LLVM Language Reference Manual. https://llvm.org/docs/LangRef.html. Accessed: 2022-10-19.

[45] Florian Lonsing, Subhasish Mitra, and Clark W. Barrett. 2020. A Theoretical Framework for Symbolic Quick Error Detection. In *2020 Formal Methods in Computer Aided Design, FMCAD 2020, Haifa, Israel, September 21-24, 2020*. IEEE, 1–10. https://doi.org/10.34727/2020/ISBN.978-3-85448-042-6_9

[46] Jiacheng Ma, Majd Ganaiem, Madeline Burbage, Theo Gregersen, Rachel McAmis, Freddy Gabbay, and Baris Kasikci. 2025. Proactive Runtime Detection of Aging-Related Silent Data Corruptions: A Bottom-Up Approach. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4* (Hilton La Jolla Torrey Pines, La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 220–235. https://doi.org/10.1145/3622781.3674182

[47] S.C. Ma, P. Franco, and E.J. McCluskey. 1995. An experimental chip to evaluate test techniques experiment results. In *Proceedings of 1995 IEEE International Test Conference (ITC)*.

[48] Timothy C. May and Murray H. Woods. 1978. A New Physical Mechanism for Soft Errors in Dynamic Memories. In *16th International Reliability Physics Symposium*. 33–40. https://doi.org/10.1109/IRPS.1978.362815

[49] E.J. McCluskey. 1993. Quality and single-stuck faults. In *Proceedings of IEEE International Test Conference - (ITC)*.

[50] E.J. McCluskey and Chao-Wen Tseng. 2000. Stuck-fault tests vs. actual defects. In *Proceedings International Test Conference 2000 (IEEE Cat. No.00CH37159)*.

[51] Yixuan Mei, Shreya Varshini, Harish Dixit, Sriram Sankar, and K. V. Rashmi. 2026. SEVI: Silent Data Corruption of Vector Instructions in Hyper-Scale Datacenters. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (USA) *(ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1711–1726. https://doi.org/10.1145/3779212.3790217

[52] Justin Meza, Qiang Wu, Sanjeev Kumar, and Onur Mutlu. 2015. Revisiting Memory Errors in Large-Scale Production Data Centers: Analysis and Modeling of New Trends from the Field. In *2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. 415–426. https://doi.org/10.1109/DSN.2015.57

[53] Subhasish Mitra, Subho Banerjee, Martin Dixon, Rama Govindaraju, Peter Hochschild, Eric X. Liu, Bharath Parthasarathy, and Parthasarathy Ranganathan. 2025. Silent Data Corruption by 10x Test Escapes Threatens Reliable Computing. arXiv:2508.01786 [cs.AR] https://arxiv.org/abs/2508.01786

[54] S. Mukherjee. 2008. *Architecture Design for Soft Errors*. https://doi.org/10.1016/B978-0-12-369529-1.X5001-0

[55] S.S. Mukherjee, J. Emer, and S.K. Reinhardt. 2005. The soft error problem: an architectural perspective. In *11th International Symposium on High-Performance Computer Architecture*.

[56] S.S. Mukherjee, C. Weaver, J. Emer, S.K. Reinhardt, and T. Austin. 2003. A systematic methodology to compute the architectural vulnerability factors for a high-performance microprocessor. In *Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture, 2003. MICRO-36*. 29–40. https://doi.org/10.1109/MICRO.2003.1253181

[57] N. Oh, S. Mitra, and E.J. McCluskey. 2002. ED4I: error detection by diverse data and duplicated instructions. *IEEE Trans. Comput.* 51, 2 (2002), 180–199. https://doi.org/10.1109/12.980007

[58] N. Oh, P.P. Shirvani, and E.J. McCluskey. 2002. Control-flow checking by software signatures. *IEEE Transactions on Reliability* 51, 1 (2002), 111–122. https://doi.org/10.1109/24.994926

[59] Nahmsuk Oh, Philip Shirvani, and Edward McCluskey. 2002. Error detection by duplicated instructions in super-scalar processors. *IEEE Transactions on Reliability* 51, 1 (2002), 63–75. https://doi.org/10.1109/24.994913

[60] OpenHW Group. 2019. CVA6 RISC-V CPU. https://github.com/openhwgroup/cva6.

[61] openssl 2024. OpenSSL. https://github.com/openssl/openssl.

[62] openssl-manual [n. d.]. OPENSSL Debian Manpages. https://manpages.debian.org/testing/libssl-doc/OPENSSL_LH_doall_arg.3ssl.en.html.

[63] George Papadimitriou and Dimitris Gizopoulos. 2023. AVGI: Microarchitecture-Driven, Fast and Accurate Vulnerability Assessment. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 935–948. https://doi.org/10.1109/HPCA56546.2023.10071105

[64] George Papadimitriou and Dimitris Gizopoulos. 2023. Silent Data Corruptions: Microarchitectural Perspectives. *IEEE Trans. Comput.* 72, 11 (2023), 3072–3085. https://doi.org/10.1109/TC.2023.3285094

[65] George Papadimitriou, Dimitris Gizopoulos, Harish Dattatraya Dixit, and Sriram Sankar. 2023. Silent Data Corruptions: The Stealthy Saboteurs of Digital Integrity.

2023 IEEE 29th International Symposium on On-Line Testing and Robust System Design (IOLTS) (2023), 1–7. https://api.semanticscholar.org/CorpusID:261315246

[66] Priyadarsan Patra. 2007. On the cusp of a validation wall. *IEEE Design & Test of Computers* 24, 2 (2007), 193–196. https://doi.org/10.1109/MDT.2007.54

[67] Bipul C. Paul, Kunhyuk Kang, Haldun Kufluoglu, Muhammad A. Alam, and Kaushik Roy. 2007. Negative Bias Temperature Instability: Estimation and Design for Improved Reliability of Nanoscale Circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 26, 4 (2007), 743–751. https://doi.org/10.1109/TCAD.2006.884870

[68] Mahesh Prabhu and Jacob A. Abraham. 2012. Functional test generation for hard to detect stuck-at faults using RTL model checking. In *2012 17th IEEE European Test Symposium (ETS)*.

[69] G.A. Reis, J. Chang, N. Vachharajani, R. Rangan, and D.I. August. 2005. SWIFT: software implemented fault tolerance. In *International Symposium on Code Generation and Optimization*. 243–254. https://doi.org/10.1109/CGO.2005.34

[70] Matthias Sauer, Young Moon Kim, Jun Seomun, Hyung-Ock Kim, Kyung-Tae Do, Jung Yun Choi, Kee Sup Kim, Subhasish Mitra, and Bernd Becker. 2013. Early-life-failure detection using SAT-based ATPG. In *2013 IEEE International Test Conference (ITC)*. 1–10. https://doi.org/10.1109/TEST.2013.6651925

[71] Jian Shen and Jacob A. Abraham. 1998. Native mode functional test generation for processors with applications to self test and design validation. *Proceedings International Test Conference 1998 (IEEE Cat. No.98CH36270)* (1998), 990–999. https://api.semanticscholar.org/CorpusID:14132281

[72] Eshan Singh, Clark W. Barrett, and Subhasish Mitra. 2017. E-QED: Electrical Bug Localization During Post-silicon Validation Enabled by Quick Error Detection and Formal Methods. (2017).

[73] Gordon L. Smith. 1985. Model for Delay Faults Based upon Paths. In *International Test Conference*.

[74] Wilson Snyder, Paul Wasson, Duane Galbi, and et al. [n. d.]. *Verilator*. https://verilator.org

[75] Daniel Sorin. 2009. *Fault Tolerant Computer Architecture*. Vol. 4. https://doi.org/10.2200/S00192ED1V01Y200904CAC005

[76] T.M. Storey and W. Maly. 1990. CMOS bridging fault detection. In *Proceedings. International Test Conference 1990*.

[77] E. Takeda and N. Suzuki. 1983. An empirical model for device degradation due to hot-carrier injection. *IEEE Electron Device Letters* 4, 4 (1983), 111–113. https://doi.org/10.1109/EDL.1983.25667

[78] Arman Vassighi, Richard Kacprowicz, Carlos Carranza, and Walter Riordan. 2008. Characterizing infant mortality in high volume manufacturing. In *2008 IEEE International Reliability Physics Symposium*. 717–718. https://doi.org/10.1109/RELPHY.2008.4559004

[79] R. L. Wadsack. 1978. Fault modeling and logic simulation of CMOS and MOS integrated circuits. *The Bell System Technical Journal* 57, 5 (1978), 1449–1474. https://doi.org/10.1002/j.1538-7305.1978.tb02106.x

[80] Ilya Wagner and Valeria Bertacco. 2008. Reversi: Post-silicon validation system for modern microprocessors. In *2008 IEEE International Conference on Computer Design*. 307–314. https://doi.org/10.1109/ICCD.2008.4751878

[81] John A. Waicukauski, Eric Lindbloom, Barry K. Rosen, and Vijay S. Iyengar. 1987. Transition Fault Simulation. *IEEE Design & Test of Computers* 4, 2 (1987), 32–38. https://doi.org/10.1109/MDT.1987.295104

[82] Shaobu Wang, Guangyan Zhang, Junyu Wei, Yang Wang, Jiesheng Wu, and Qingchao Luo. 2023. Understanding Silent Data Corruptions in a Large Production CPU Population. In *Proceedings of the 29th Symposium on Operating Systems Principles*.

[83] Shaobu Wang, Guangyan Zhang, Junyu Wei, Yang Wang, Jiesheng Wu, and Qingchao Luo. 2024. Understanding Silent Data Corruption in Processors for Mitigating its Effects. *ACM Trans. Archit. Code Optim.* 21, 4, Article 84 (Nov. 2024), 27 pages. https://doi.org/10.1145/3690825

[84] James F Ziegler and William A Lanford. 1979. Effect of cosmic rays on computer memories. *Science* 206, 4420 (1979), 776–788.

[85] zlib 2024. Zlib. https://zlib.net/.